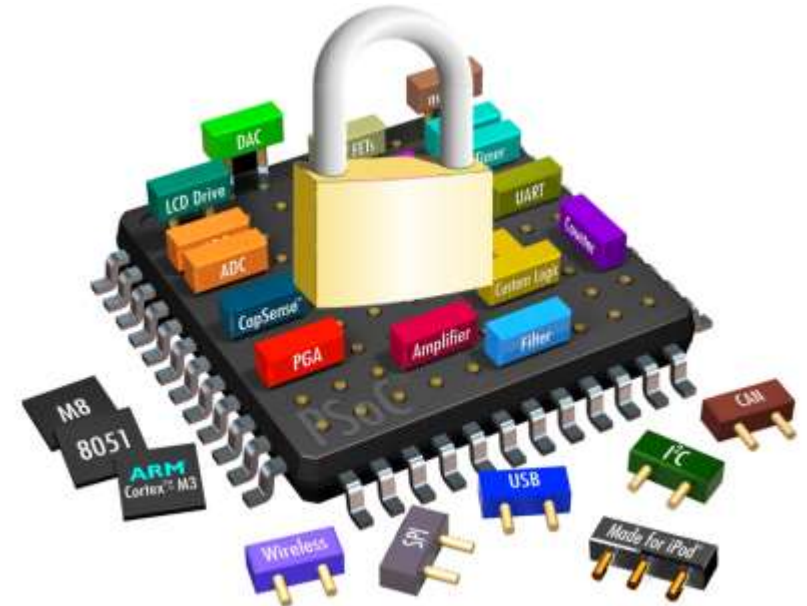


Hardware Security

TEE Examples: AMD SEV



SEV Evolution

1**2016: SEV (Secure Encrypted Virtualization)**

First x86 technology to isolate VMs from hypervisor using main memory encryption. Each VM gets unique AES encryption key for automatic data protection.

2**2017: SEV-ES (Encrypted State)**

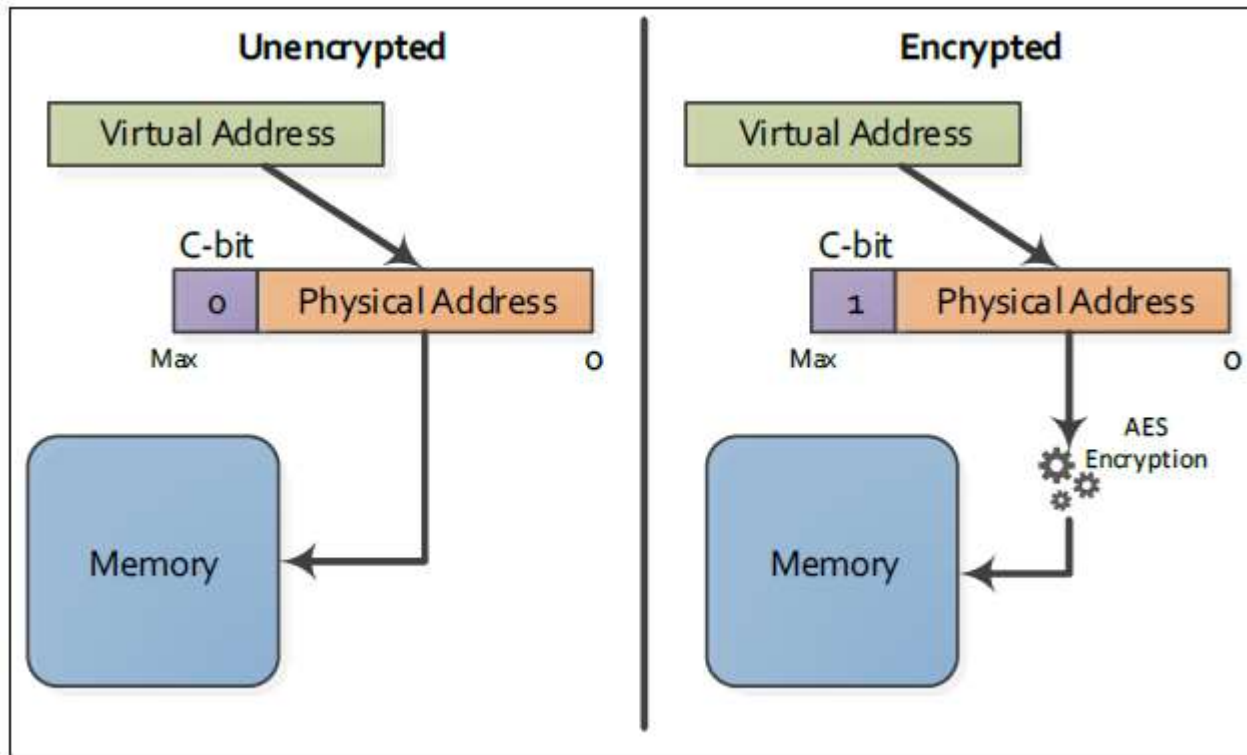
Added protection for CPU register state. VM register state encrypted on each hypervisor transition, preventing hypervisor from seeing actively used data.

3**2020: SEV-SNP (Secure Nested Paging)**

Introduces strong memory integrity protection to prevent malicious hypervisor attacks like data replay, memory re-mapping, and corruption.



Private vs. Shared Memory



- VMs control memory page status using the enCRypted bit (C-bit) in guest page tables. The C-bit location is implementation-defined and may be the top physical address bit.
- This architecture enables VMs to selectively share data while maintaining strong protection for private information.

Threat Model

■ Trusted Components

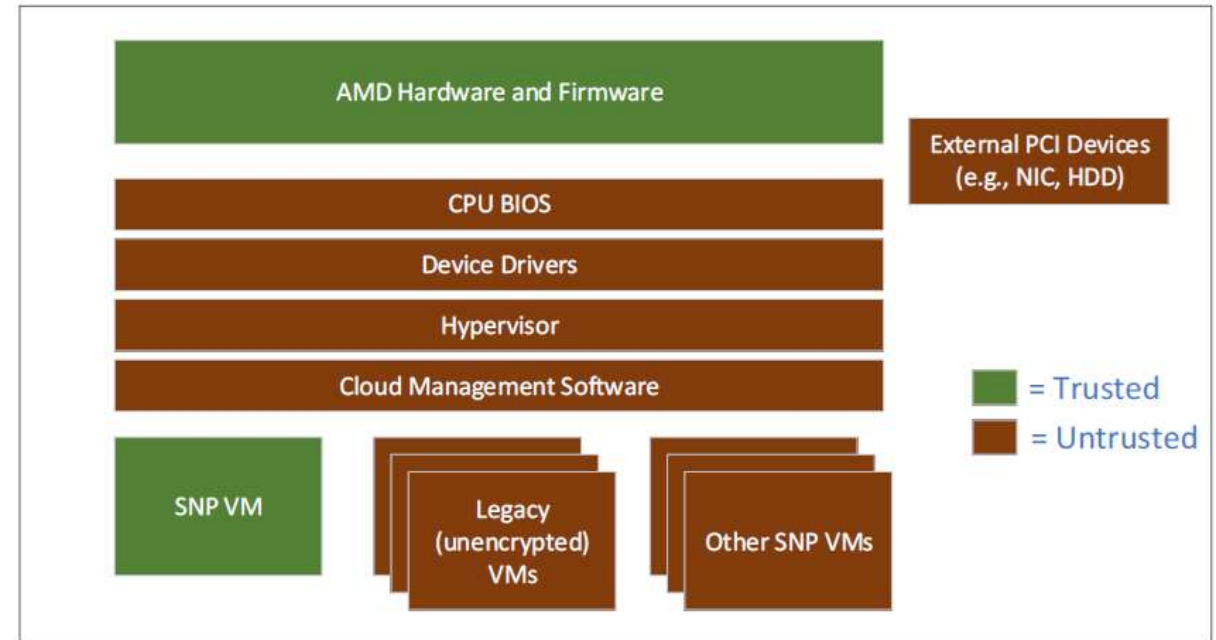
- AMD System-On-Chip hardware, AMD Secure Processor, and the VM itself are fully trusted. VM must protect its own interfaces following security best practices.

■ Untrusted Components

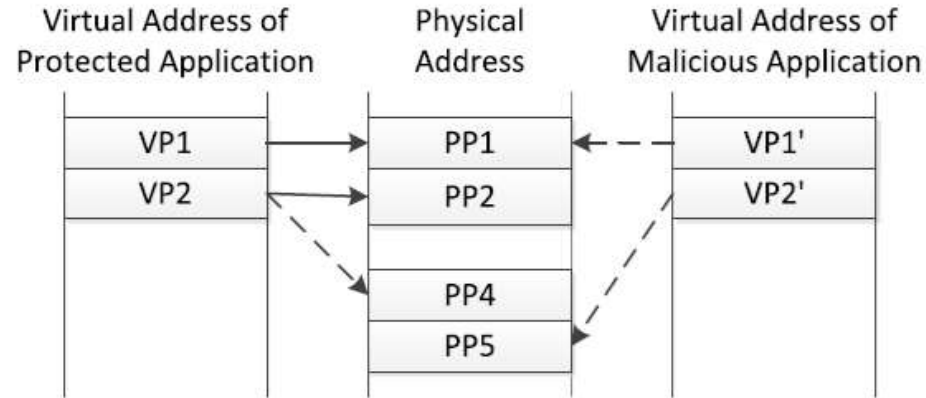
- All other CPU software and PCI devices treated as fully untrusted including BIOS, hypervisor, device drivers, and other VMs, assumed potentially malicious.

■ Protection Scope

- SEV-SNP protects against confidentiality and integrity attacks, maintains hypervisor availability, and includes TCB rollback protection and optional interrupt/side-channel defenses.



Memory Mapping Threats



- Data Corruption
 - Only the owner of a memory page can write that page
- Memory Aliasing
 - Every physical memory page can map only to a single guest page at one time
- Memory Re-mapping
 - Every guest page can map only to a single physical memory page at one time

Reverse Map Table

■ Inverted Page Table

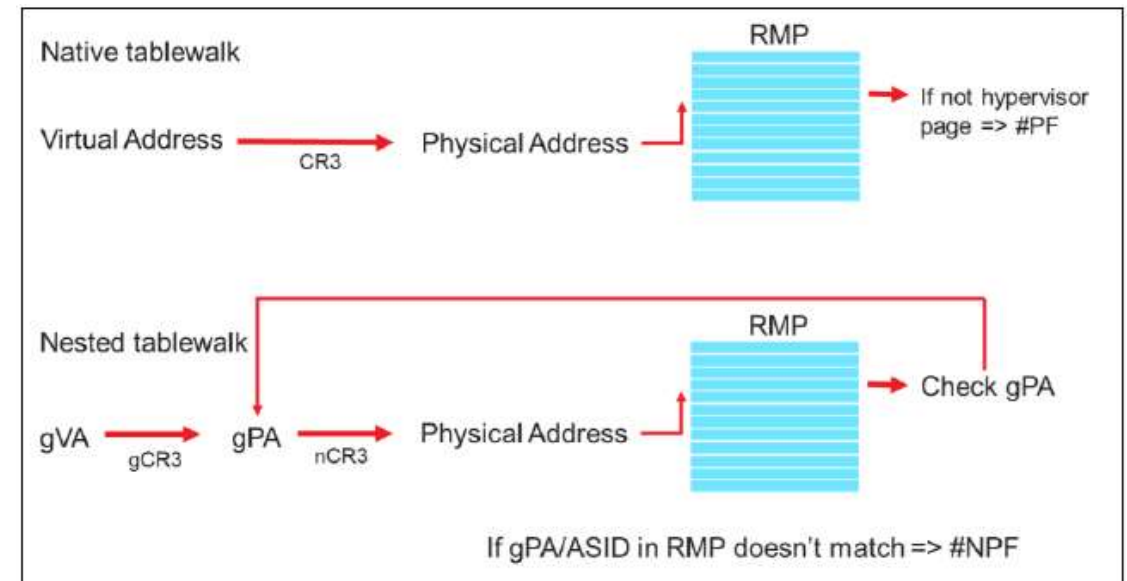
- Mapping physical to virtual addresses

■ Native Mode

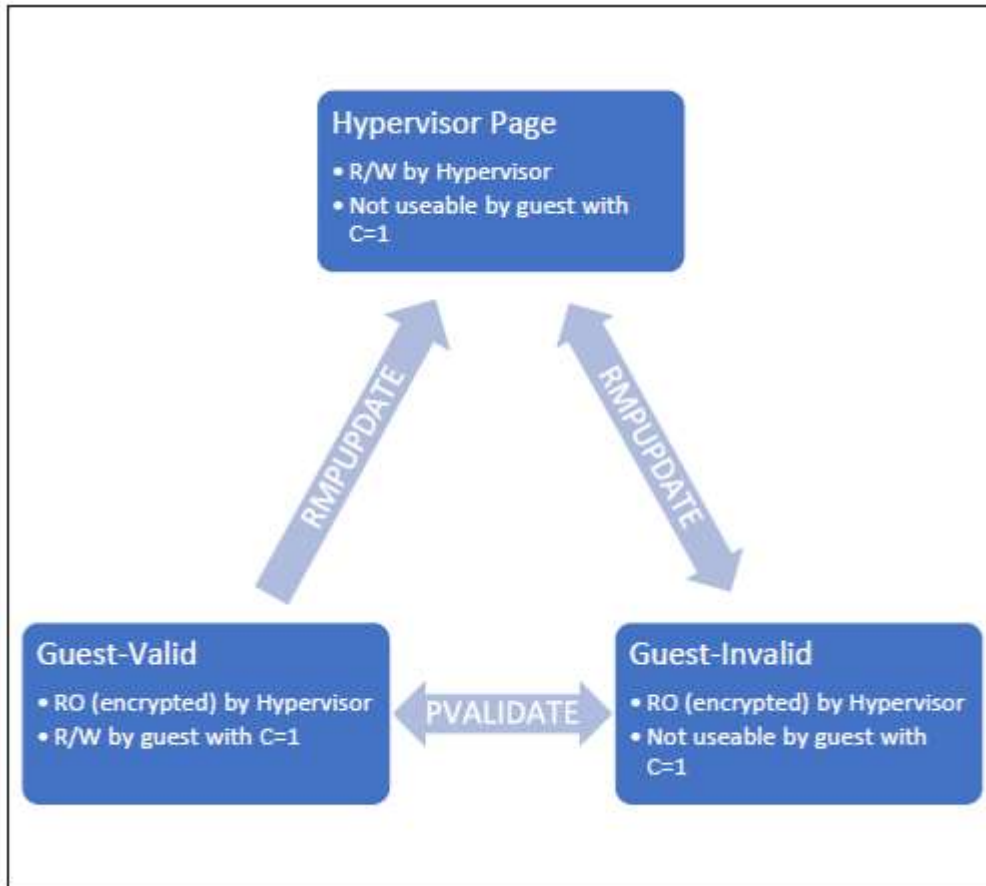
- Virtual addresses translated to physical addresses using standard x86 page tables. Final physical address indexes RMP. If page is hypervisor-owned, checks pass and TLB entry created. Otherwise, table-walk faults.

■ SEV-SNP VM Mode

- Guest Virtual Address translated to Guest Physical Address, then to System Physical Address using 2-level paging. SPA indexes RMP. Entry must show page is guest-owned, assigned to this specific guest, and mapped at correct GPA.



Page Validation



Each RMP entry contains a Validated bit, automatically cleared to 0 when new RMP entry is created for a guest. Pages with Validated bit clear cannot be used as private guest pages until the guest sets the bit using the PVALIDATE instruction.

Hypervisor Assignment

Hypervisor assigns page to guest using RMPUPDATE instruction, transitioning page to Guest-Invalid state

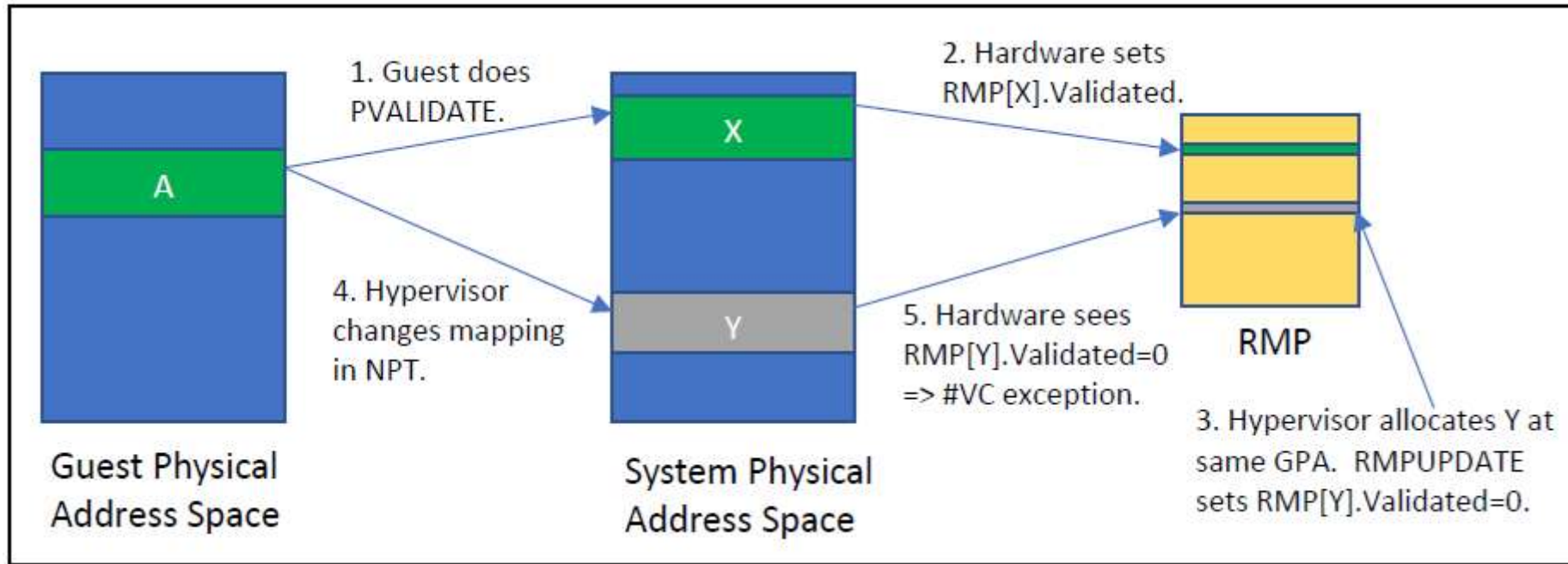
Guest Validation

Guest validates page using PVALIDATE instruction, transitioning to Guest-Valid state where it can be used

One-Time Rule

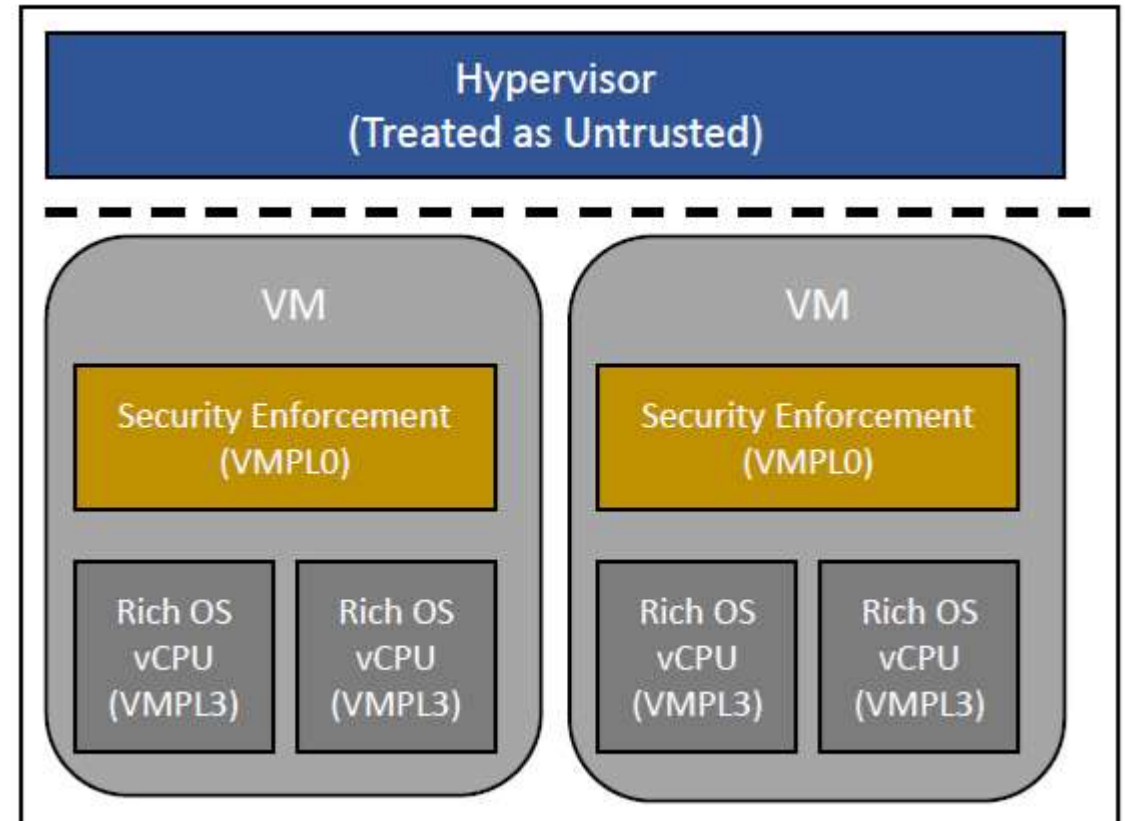
Guest VM should never validate same GPA more than once, ensuring injective mapping between GPAs and SPAs

Page Re-mapping Attack



Virtual Machine Privilege Levels (VMPLs)

- VMPLs are an optional feature allowing guest VMs to divide their address space into four hierarchical levels.
- VMPL0 is highest privilege, VMPL3 is least privileged.
- This enables hardware-isolated abstraction layers within a VM for additional security controls.



VMPL Use Cases

Secure Hypervisor Virtualization

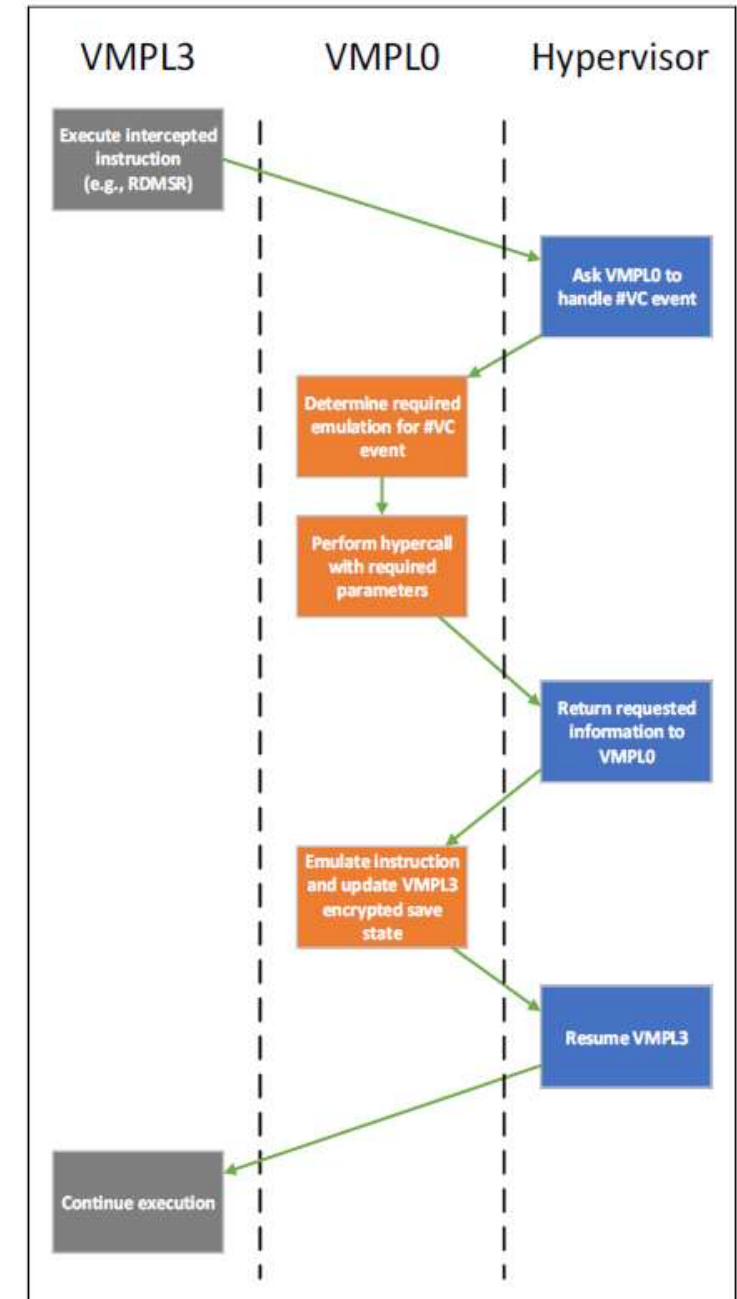
VMPL0 enforces page permissions like a bare-metal hypervisor would, enabling secure virtualization of security-enforcing hypervisors in untrusted cloud environments.

Trusted APIC Emulation

VMPL0 performs trusted APIC emulation inside guest trust domain, while rest of guest runs in lower VMPLs unaware of emulation.

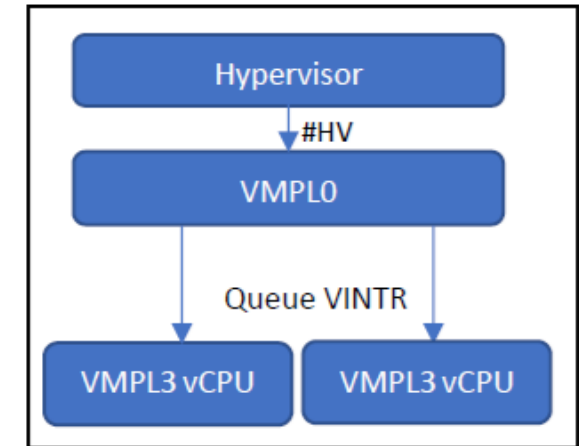
Legacy Guest Support

VMPL0 acts as intermediary for guest-hypervisor communications, handling #VC exceptions and C-bit management, enabling unenlightened legacy VMs to use SEV-SNP.



Interrupt and Exception Protection

- SEV-SNP adds two optional modes for more restrictive interrupt/exception interfaces between VM and hypervisor, protecting against violations of OS assumptions about interrupt behavior.
- Restricted Injection
 - Disables virtual interrupt queuing and partially disables interrupt injection interface.
 - Hypervisor can only inject new #HV exception vector as doorbell.
 - VM and hypervisor communicate via para-virtualized event queue in shared memory.
- Alternate Injection
 - Allows standard virtual interrupt queuing and injection, but only controlled by guest itself.
 - New fields in encrypted Virtual Machine Save Area (VMSA) allow interrupt queuing and event injection.
 - All interrupt-related security-sensitive state saved to VMSA.

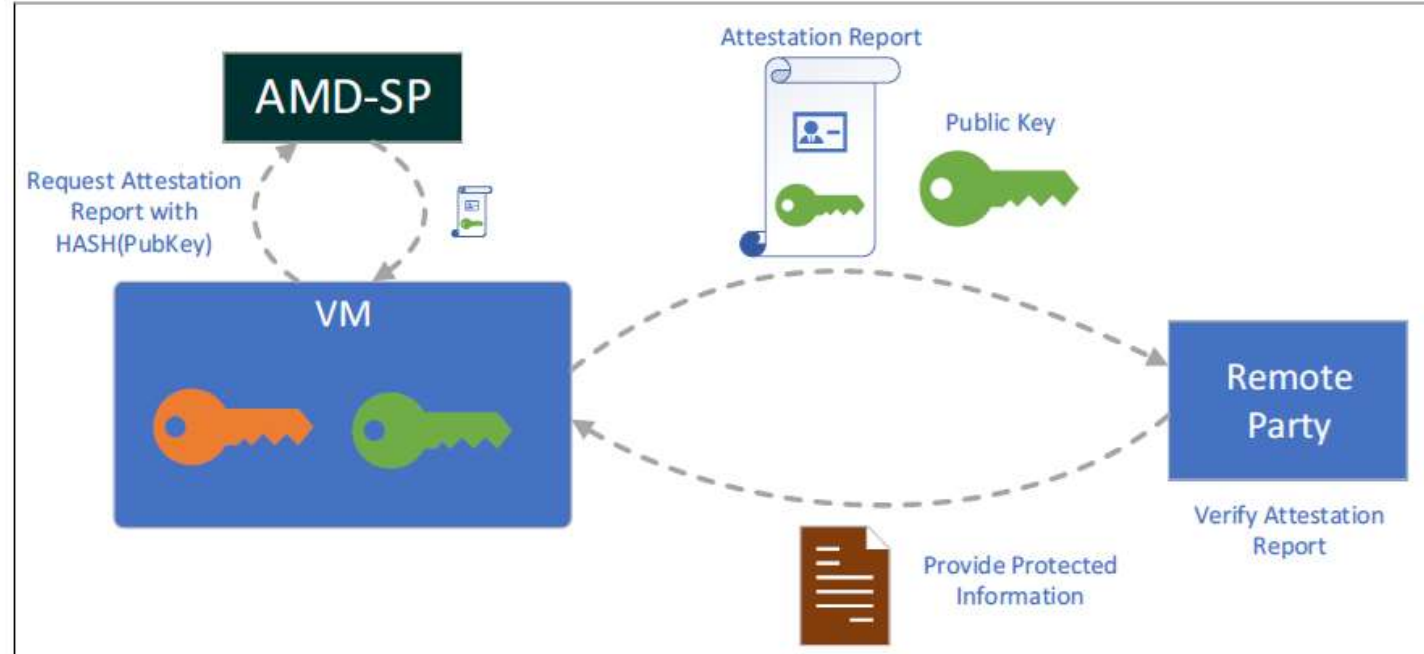


VM Launch Process

■ Process

- SEV-SNP VMs start from initial unencrypted image containing boot code but no secrets.
 - Hypervisor asks AMD-SP (AMD Secure Processor) to install initial pages.
 - AMD-SP cryptographically measures contents and metadata into launch digest.
- ## ■ Guest owner supplies signed Identity Block (IDB) associating with VM.
- IDB contains fields to uniquely identify VM and expected launch digest.
 - Only VMs matching expected digest can be associated with IDB.

Attestation Process



- Unlike SEV/SEV-ES, SEV-SNP supports attestation at any time through protected path

VM Migration

SEV-SNP enhances migration flexibility by offloading policy enforcement to a Migration Agent (MA)—itself an SEV-SNP VM running on same physical system as primary VM. Each VM can be associated with one MA, but single MA can manage migration for arbitrary number of VMs.

Complex Policy Implementation

MA determines what systems primary VM can migrate to, implementing complex policies using any desired means. MA information included in VM attestation report as part of TCB.

Flexible Migration Timing

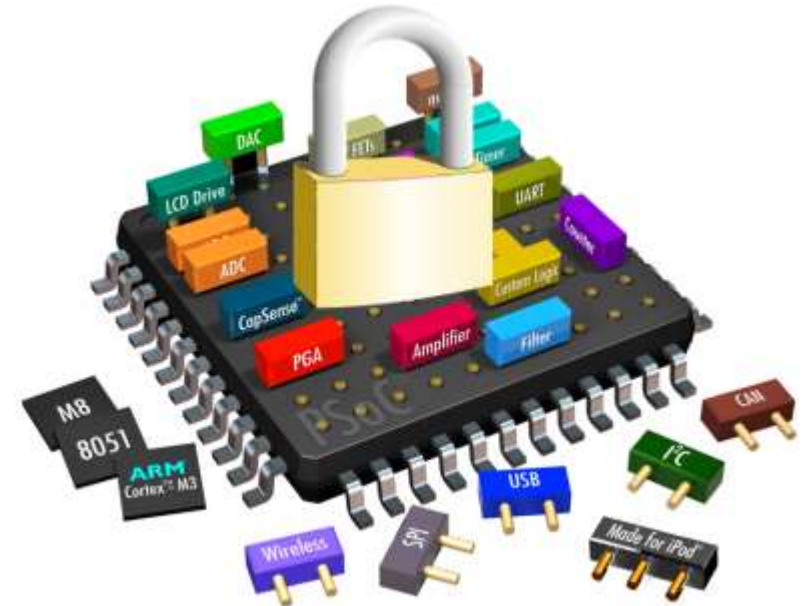
Source and destination machines don't need to be online simultaneously. MA can hold VM state or put into long-term storage, reconstituting VM later on same or different machine.

Secure Transfer Process

MA on source machine authenticates MA on destination machine and establishes protected network connection. If successful, transfers required guest information for reconstitution.

Hardware Security

TEE Examples: Intel TDX



What is Intel TDX?

- Intel Trust Domain Extensions (TDX) is an architectural extension in 4th Generation Intel Xeon Scalable Processors that enables confidential computing.
- TDX allows deployment of **virtual machines** in Secure-Arbitration Mode (SEAM) with encrypted CPU state and memory, integrity protection, and remote attestation.
- TDX enforces hardware-assisted isolation for virtual machines and minimizes the attack surface exposed to host platforms, which are considered untrustworthy in confidential computing's threat model.

Core Security Principles

01

Memory Confidentiality

TD (Trust Domain) data is encrypted when offloaded to main memory using TD-specific cryptographic keys known only to the processor

02

CPU State Confidentiality

Virtual CPU states are protected during context switches, stored in encrypted metadata

03

Execution Integrity

Protects TD execution from host interference, detecting malicious changes in virtual CPU states

04

I/O Protection

Peripheral devices cannot access TD private memory; shared memory requires explicit TD consent

TDX Threat Model

■ Adversary Capabilities

- Physical or remote access to computer
- Control over boot firmware, SMM, host OS, hypervisor
- Control of peripheral devices
- Ability to interrupt TDs and manipulate memory
- Physical and hardware attacks on buses

■ What TDX Protects

- TDX prevents adversaries from compromising **confidentiality** and **integrity** of TD memory and CPU state.
- However, it cannot guarantee **availability** - adversaries can launch DoS attacks by controlling compute resources.

Trusted Computing Base (TCB)

Hardware Components

TDX-enabled Intel processors with VT, MKTME, and SGX technologies

Intel-Signed Software

TDX Module, NP/P-SEAM Loaders, and architectural SGX enclaves for attestation

Tenant Software

Software stacks running within TDs, owned and controlled by the tenant

Trust Requirements

Tenants must trust Intel for development, manufacturing, and signing of components, plus Intel's Provisioning Certification Service

Building Block: Intel VT

■ Virtualization Technology

- Intel VT provides hardware-assisted virtualization with better performance, isolation, and security than software-based approaches.
 - VMX root mode for hypervisor
 - VMX non-root mode for guest VMs
 - Extended Page Table (EPT) for address translation
 - VT-d for I/O device isolation

■ TDX Integration

- TDX extends VT with new SEAM VMX modes.
- Untrusted hypervisor → TDX Module
- VMX root mode → SEAM VMX root mode
- VMX non-root mode → SEAM VMX non-root mode

Building Block: Intel MKTME

1

TME Foundation

Total Memory Encryption encrypts entire memory using single transient key generated at boot-time with AES-XTS algorithm

2

MKTME Extension

Multi-key TME supports multiple keys for page-granularity encryption using Host Key Identifiers (HKIDs)

3

TDX Application

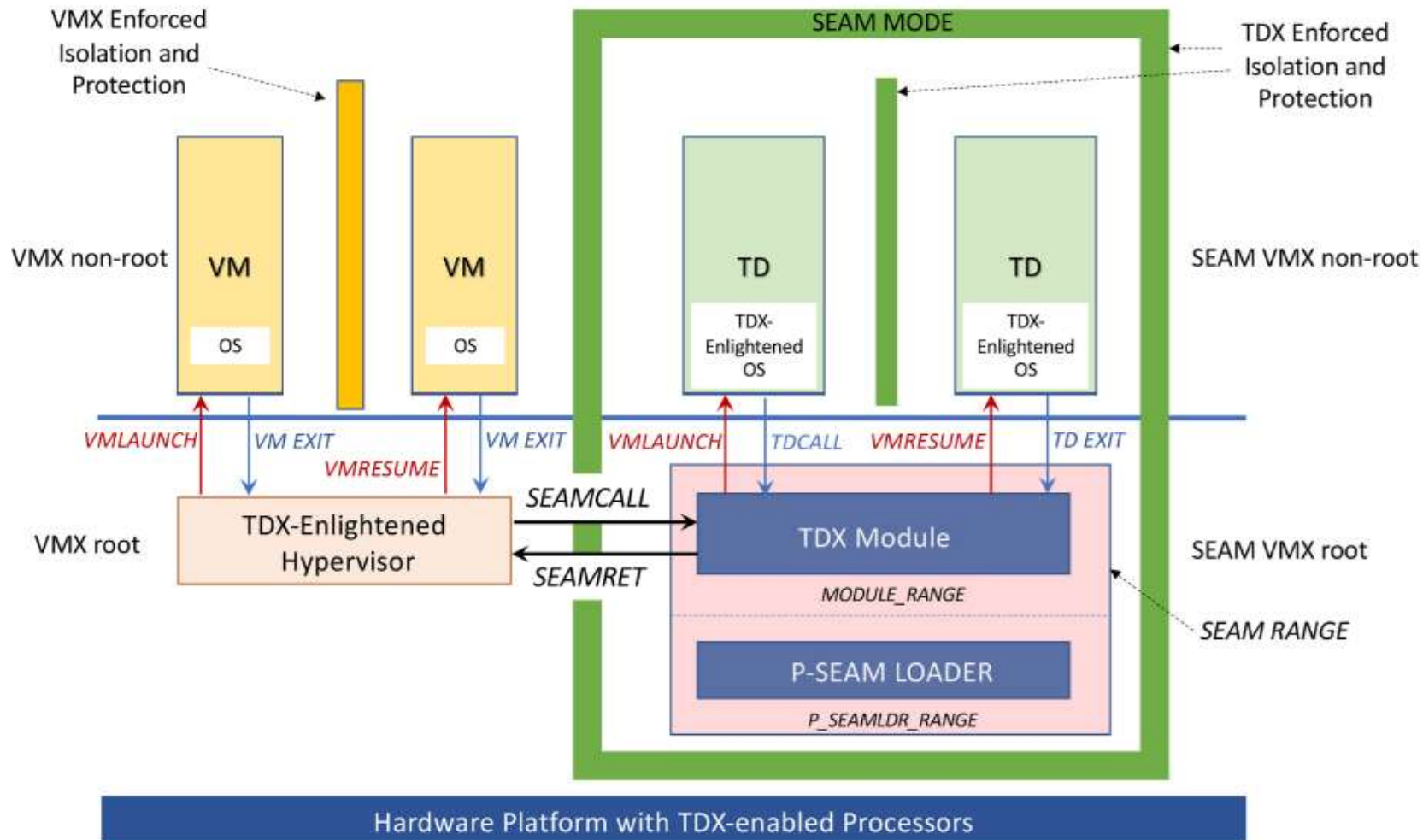
TDX Module controls memory encryption for TDs, partitioning HKID space into private and shared ranges



Building Block: Intel SGX

- SGX protects against memory bus snooping and cold boot attacks by partitioning applications into secure enclaves. Memory encryption protects enclave contents from unauthorized access.
 - Enclave Page Cache (EPC) for secure memory
 - Memory Encryption Engine (MEE)
 - Local and remote attestation
 - 18 new ISA instructions
- TDX Integration
 - TDX leverages SGX's remote attestation mechanism.
 - The Quoting Enclave verifies and signs TD reports, enabling remote verification of TD authenticity.

TDX System Architecture



- SEAM VMX Non-Root Mode
 - Trust Domains (TDs) execute in this mode with encrypted CPU state and memory
- SEAM VMX Root Mode
 - TDX Module executes in this mode, handling SEAMCALL instructions from hypervisor
- Mode Transitions
 - SEAMCALL/SEAMRET instructions enable secure transitions between modes

Memory Partitioning

■ Memory Partition

- Normal memory
- Secure memory for storing sensitive data
- Shared memory for I/O (not protected)

■ HKID Space Partitioning

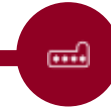
- The HKID space is partitioned at boot into private HKIDs (for TDs and TDX Module) and shared HKIDs (for legacy VMs and host)
- Each TD receives a unique private HKID bound to an ephemeral encryption key.

Memory Confidentiality and Integrity



Logical Integrity (Li)

Uses **TD Owner Bit** to detect unauthorized writes at software level.
Bit is set to 1 for TD writes, cleared to 0 for external writes.



Cryptographic Integrity (Ci)

Computes **28-bit MAC** on each cache line write using 128-bit MAC key. Recalculates MAC on read to detect tampering.

Both modes mark tampered memory as "poisoned." Reading poisoned content triggers TD exit (for TD) or disables TDX Module (for Module reads). However, neither can detect memory replay attacks if both content and metadata are rolled back.

TDX I/O Model

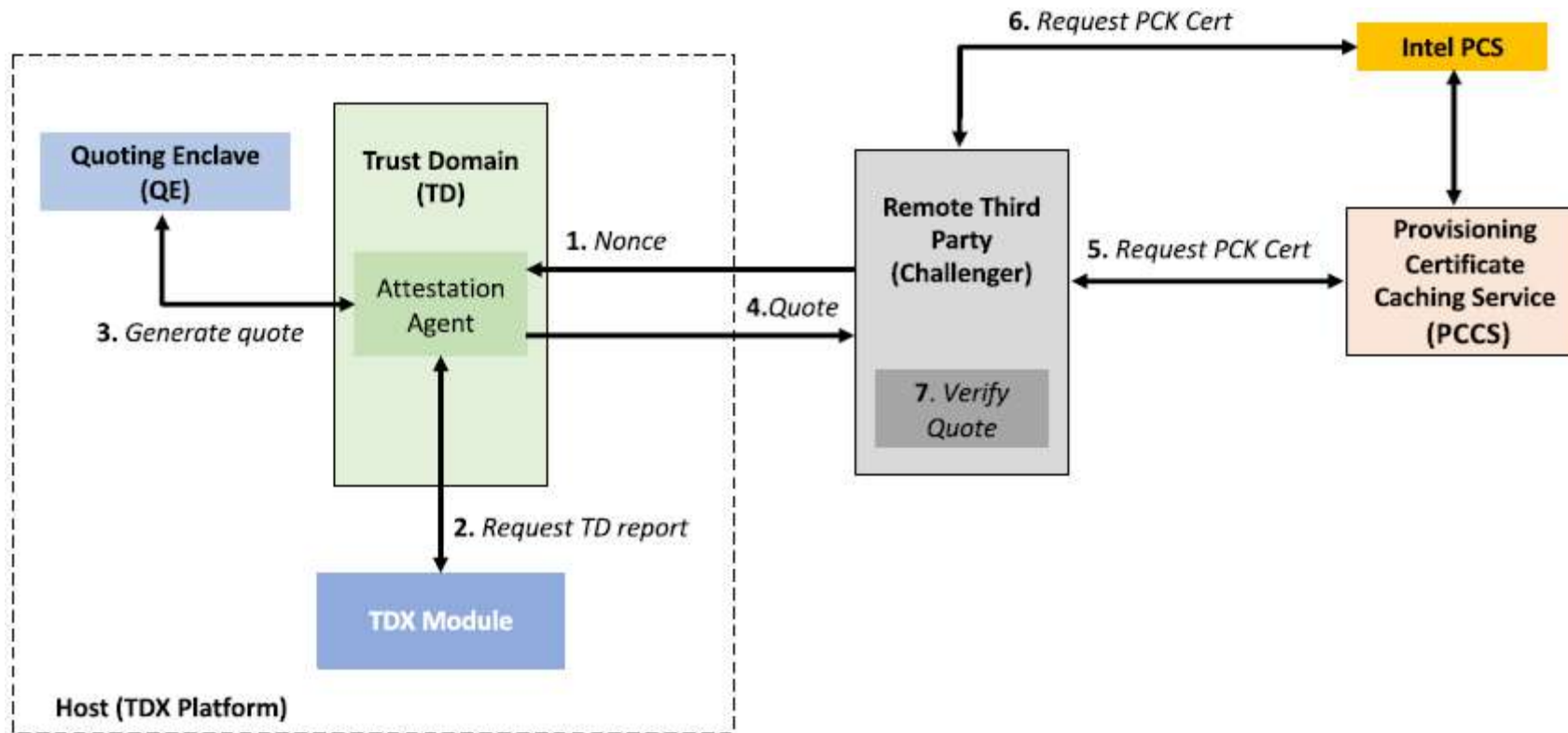
■ Security Requirements

- Hypervisors and peripheral devices are untrusted and cannot access TD private memory.
- TDs must explicitly share memory for I/O by setting the shared bit in Guest Physical Address (GPA).

■ TD Responsibilities

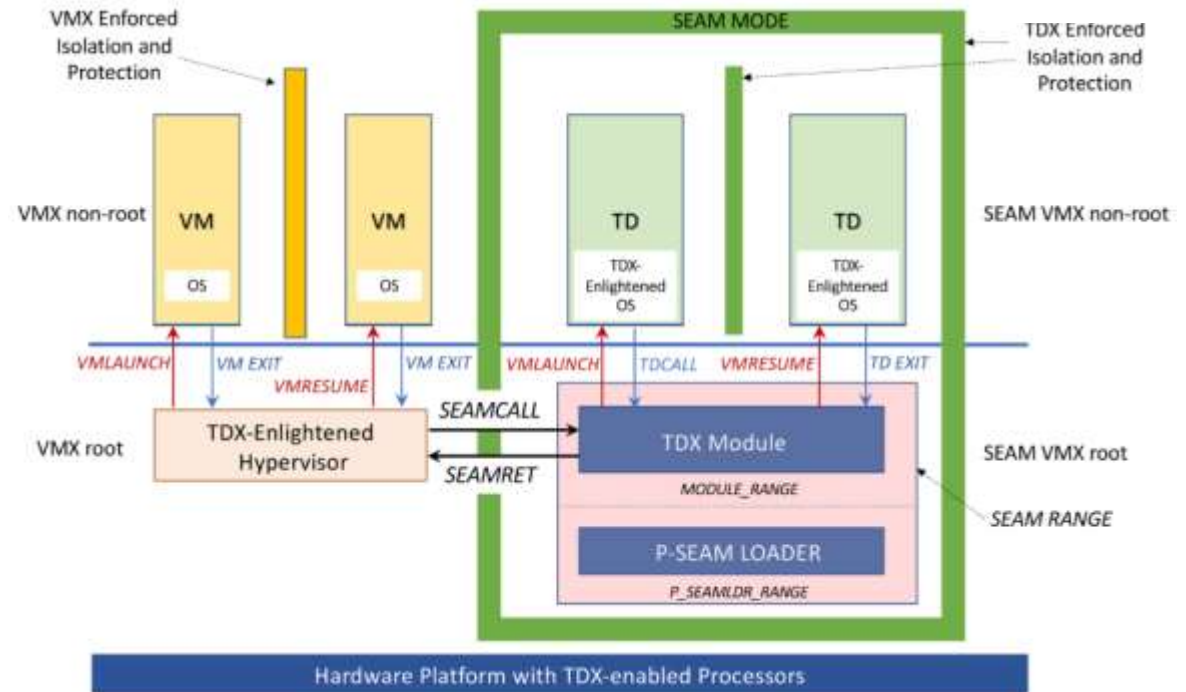
- Seal I/O data before leaving trust boundary
- Place sealed data in shared memory
- Validate all incoming I/O data
- Use hardened drivers only

Remote Attestation

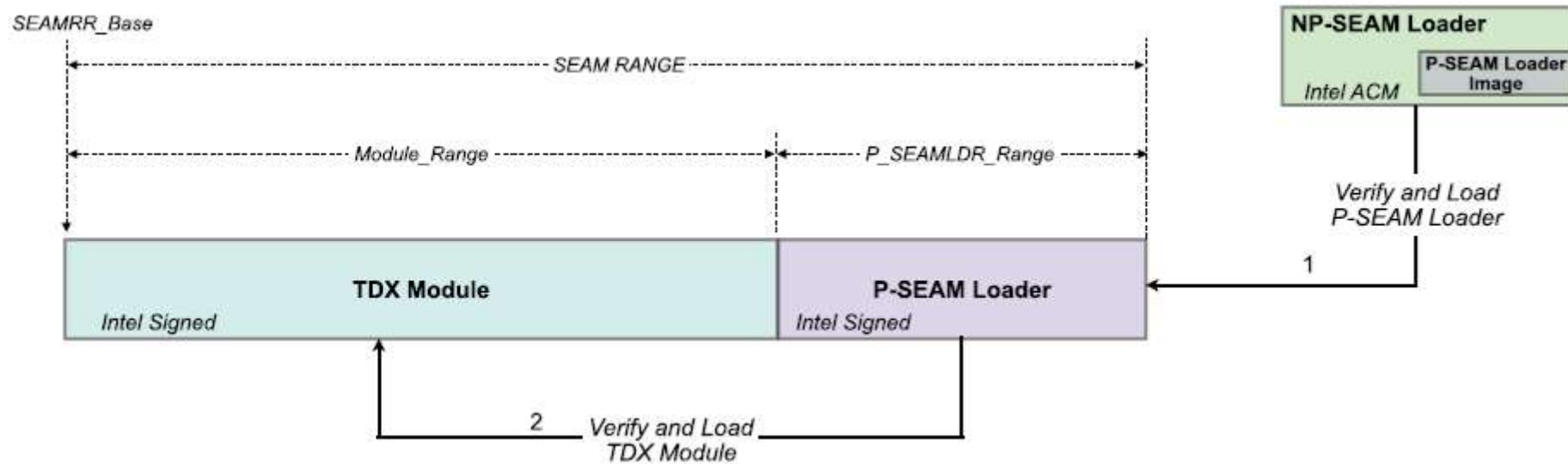


TDX Module

- TDX Module is an Intel-signed and CPU-attested software module that leverages the features of TDX-enabled processors to facilitate the construction, execution, and termination of TDs while enforcing the security guarantees.
- It provides two sets of interface functions:
 - Host-side interface functions for a TDX-enlightened hypervisor
 - Guest-side interface functions for TDs.
- It is loaded and executed in the SEAM Range, which is a portion of system memory

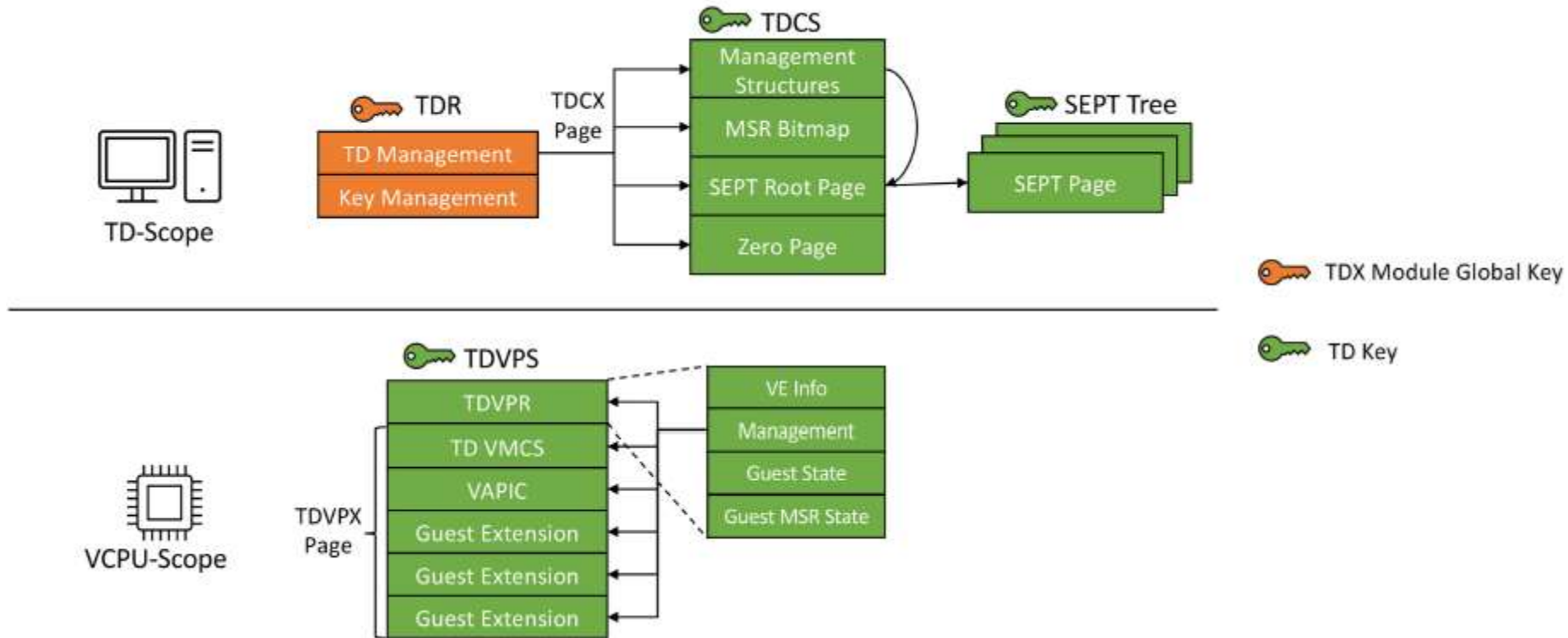


TDX Module Loading



- Intel Non-Persistent SEAM Loader (NP-SEAM Loader)
 - Intel Authenticated Code Module (ACM).
 - ACMs are Intel-signed modules that run within the internal RAM of the processor.
 - The NP-SEAM Loader is authenticated and loaded by the Intel Trusted Execution Technology (TXT)

TD Metadata



- The TDX Module is responsible for managing the entire life cycle of TDs and maintaining metadata for each TD instance.
- The TDX Module ensures that the memory encryption is applied to the metadata to prevent the hypervisor from accessing or modifying it.

Metadata Structures

TDR (Trust Domain Root)

Initial structure with encryption key info and TDCX references, protected by global private key

TDCS (Control Structure)

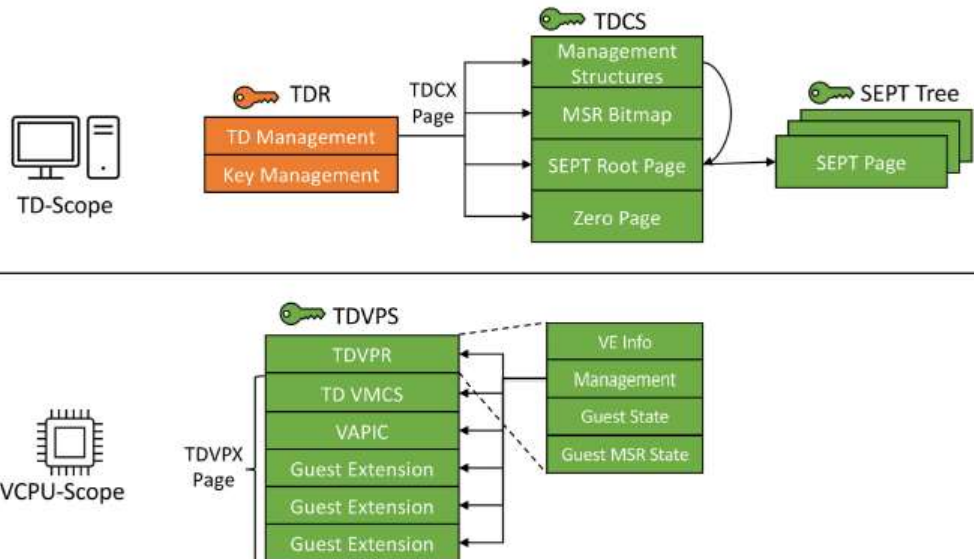
Four pages for TD management, MSR bitmaps, SEPT root, encrypted with TD private key

TDVPS (Virtual Processor State)

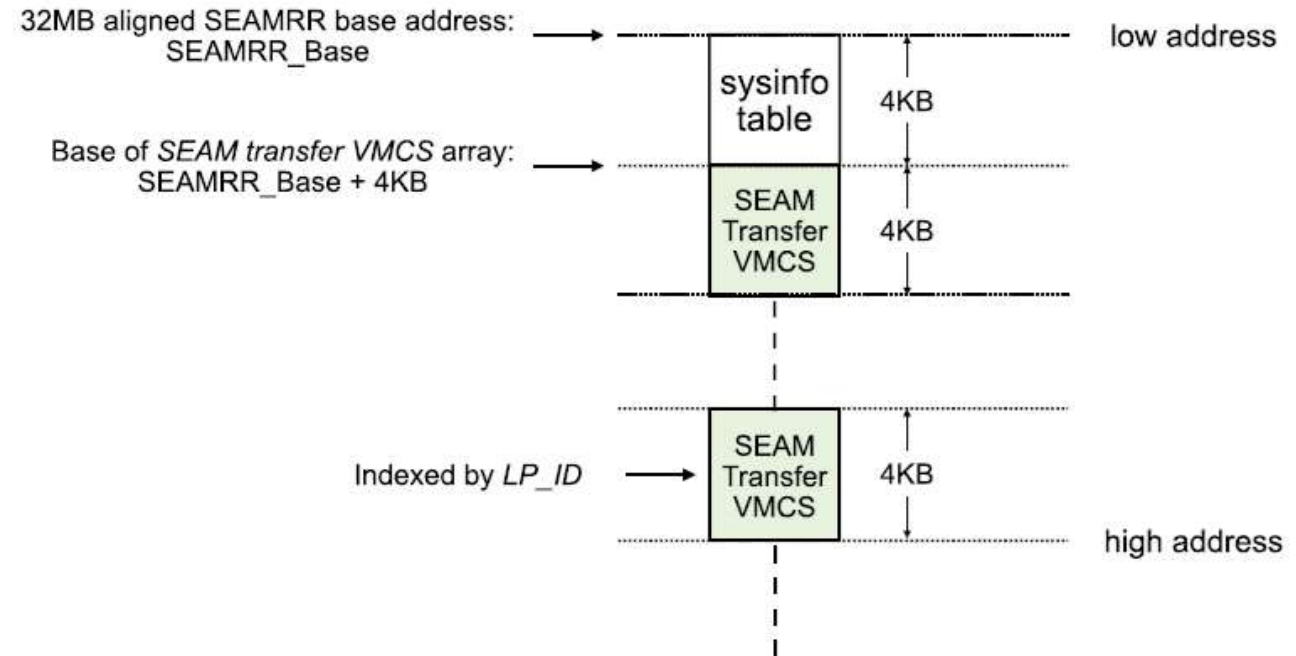
Six pages per virtual CPU: TDVPR, TD Transfer VMCS, VAPIC, guest extensions

SEPT (Secure EPT)

Protected page table for TD private memory address translation



Context Switches



■ Hypervisor ↔ TDX Module

- In TDX, a hypervisor is required to interact with the TDX Module for managing TDs through SEAMCALL interface functions.
- When SEAMCALL is made, the processor transitions from the VMX root mode to the SEAM VMX root mode and the TDX Module's SEAM Transfer VMCS (Virtual Machine Control Structure) is loaded.
- The SEAM Transfer VMCS is a means of switching the execution context between the hypervisor and the TDX Module.

Context Switches

■ TD ↔ TDX Module

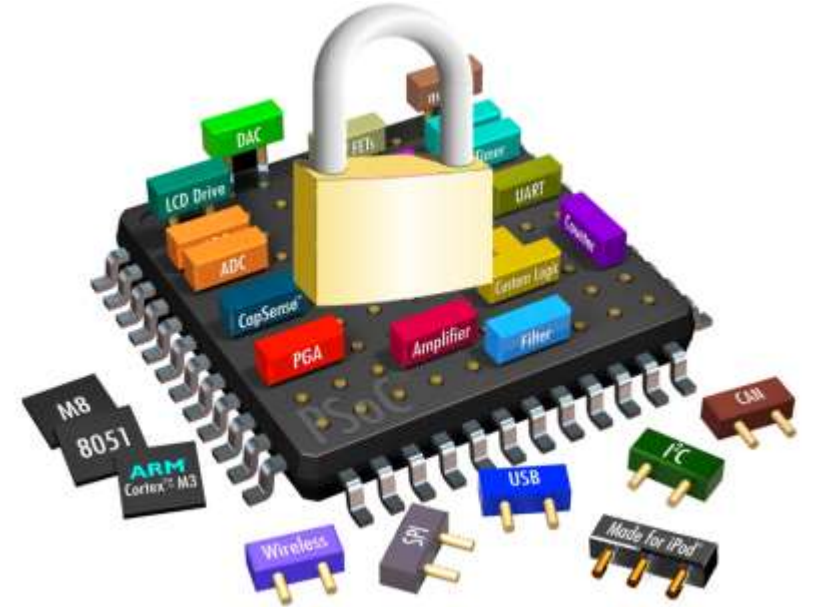
- In traditional virtualization, the hypervisor handles VM exits, which are controlled by the VM's Transfer VMCS. Each VMCS is associated with one virtual CPU and stores the virtual CPU state for recovering the guest execution in the next VM resume.
- However, this operation leaks the virtual CPU state as VMCS is visible to hypervisors.
- In TDX, synchronous TDCALLs or asynchronous TD exits are designed to trap into the TDX Module. This is controlled by the TD Transfer VMCS, which is set up when a virtual CPU of a TD is created and stored in the TD's TDVPS.
- The TDVPS is encrypted with the TD's private key. Therefore, the **TD Transfer VMCS is inaccessible to untrusted hypervisors.**

Keyholes

- All memory buffers passed through SEAMCALLs use their physical addresses as references.
- The TDX Module must map these buffers into its own linear address space to access them.
- The Keyhole region is a reserved linear address range specifically for address mapping.

Hardware Security

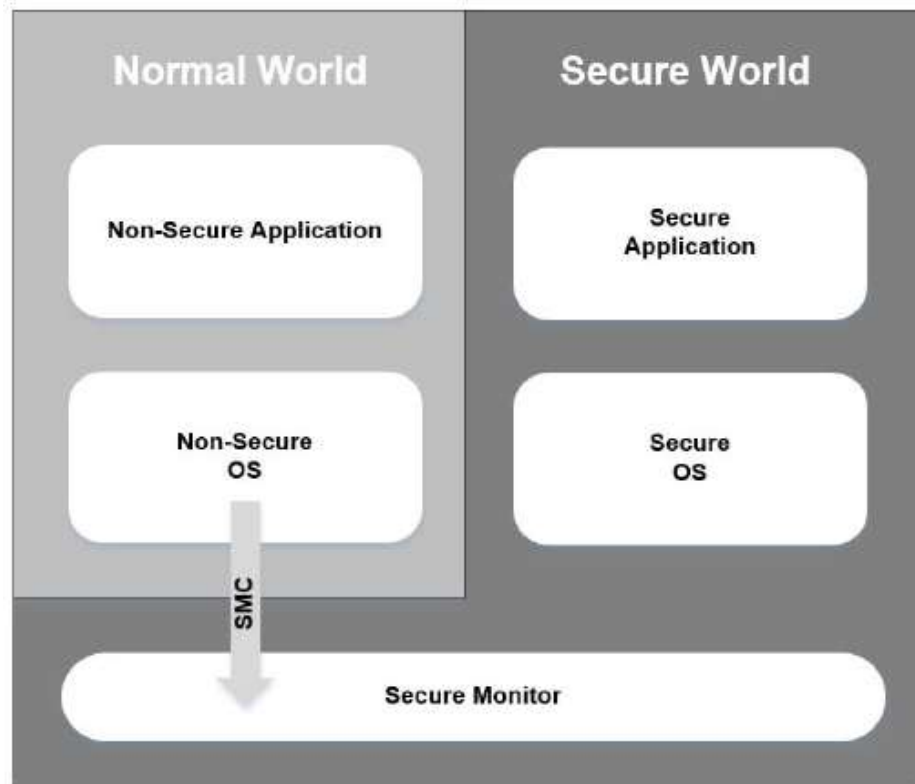
TEE Examples: ARM TrustZone



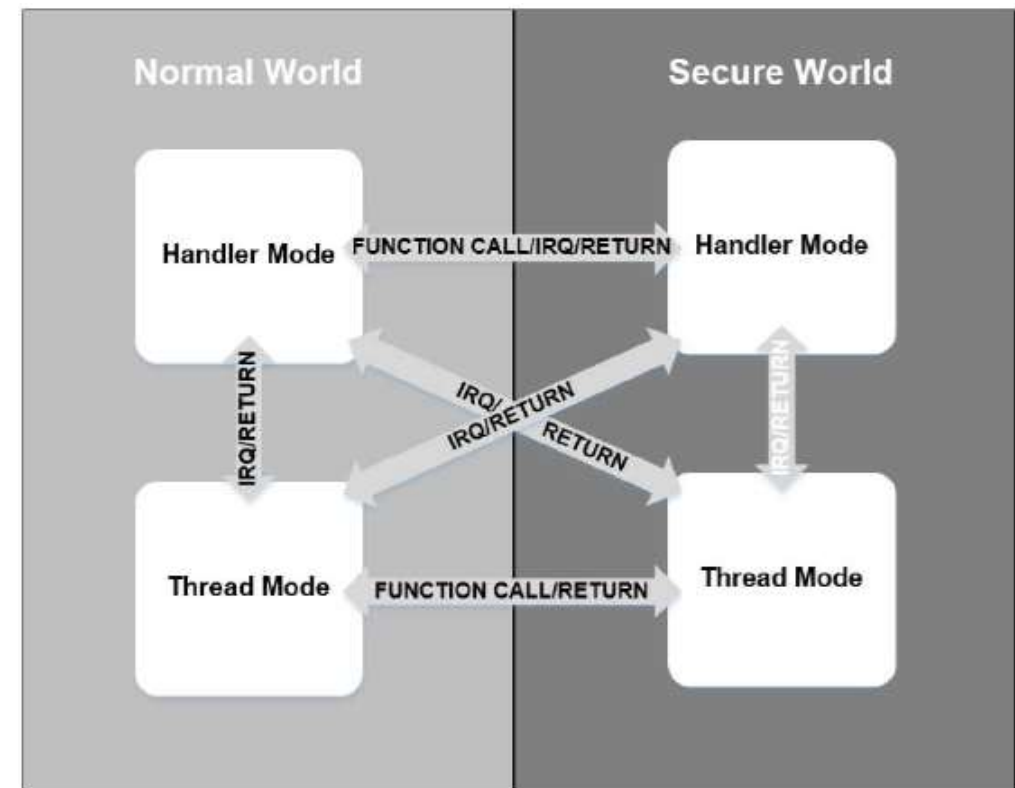
Security States

- With TrustZone Security Extensions, processors have two security states
- Secure State (Secure World)
 - Everything that runs when processor state is secure. Has access to all resources including secure memory regions.
- Non-Secure State (Normal World)
 - Everything that runs when processor is in non-secure state. Hardware barriers prevent access to secure world resources.

Software Architecture

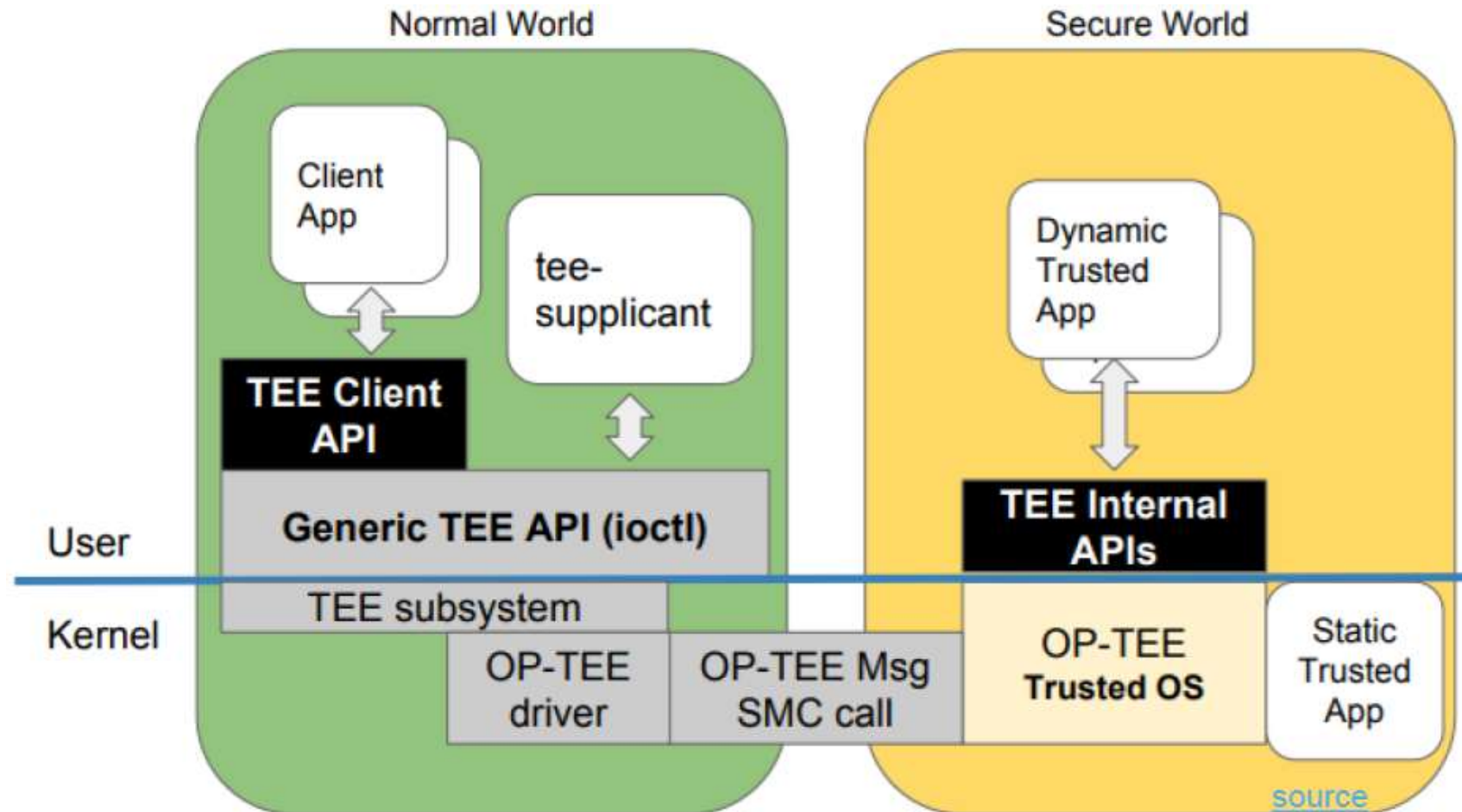


Cortex-A

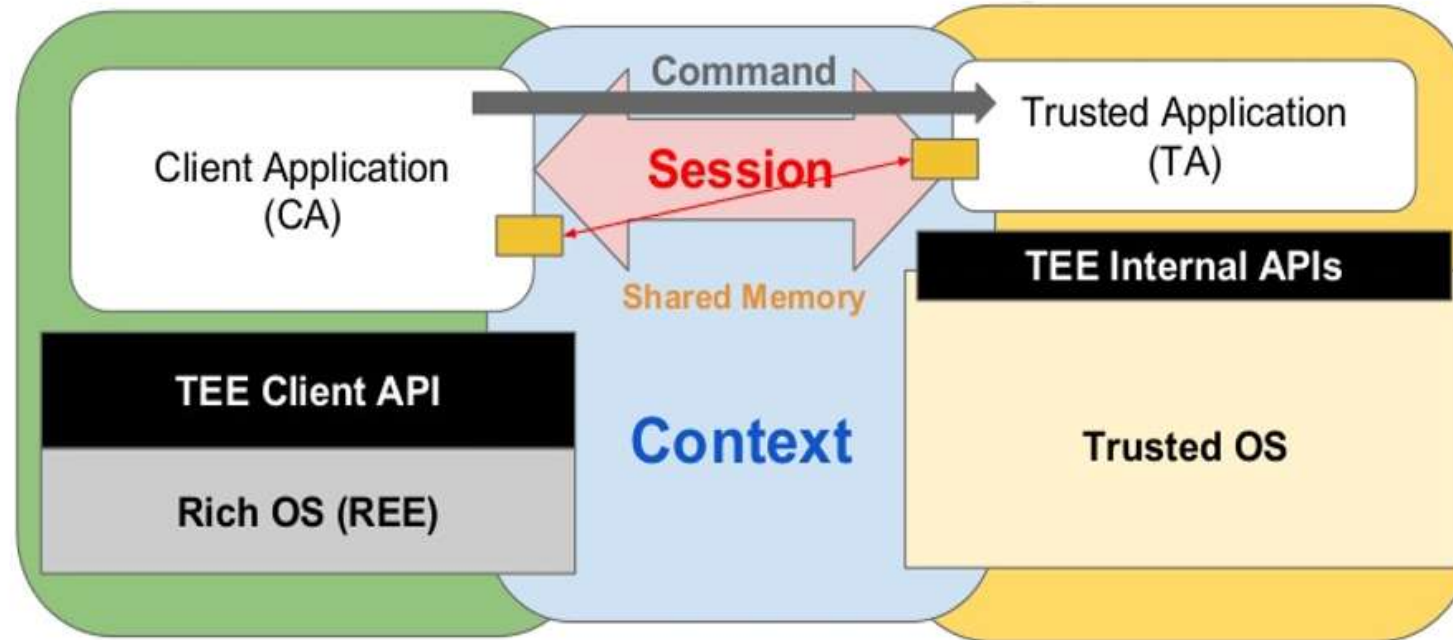


Cortex-M

OP-TEE Architecture



Communication



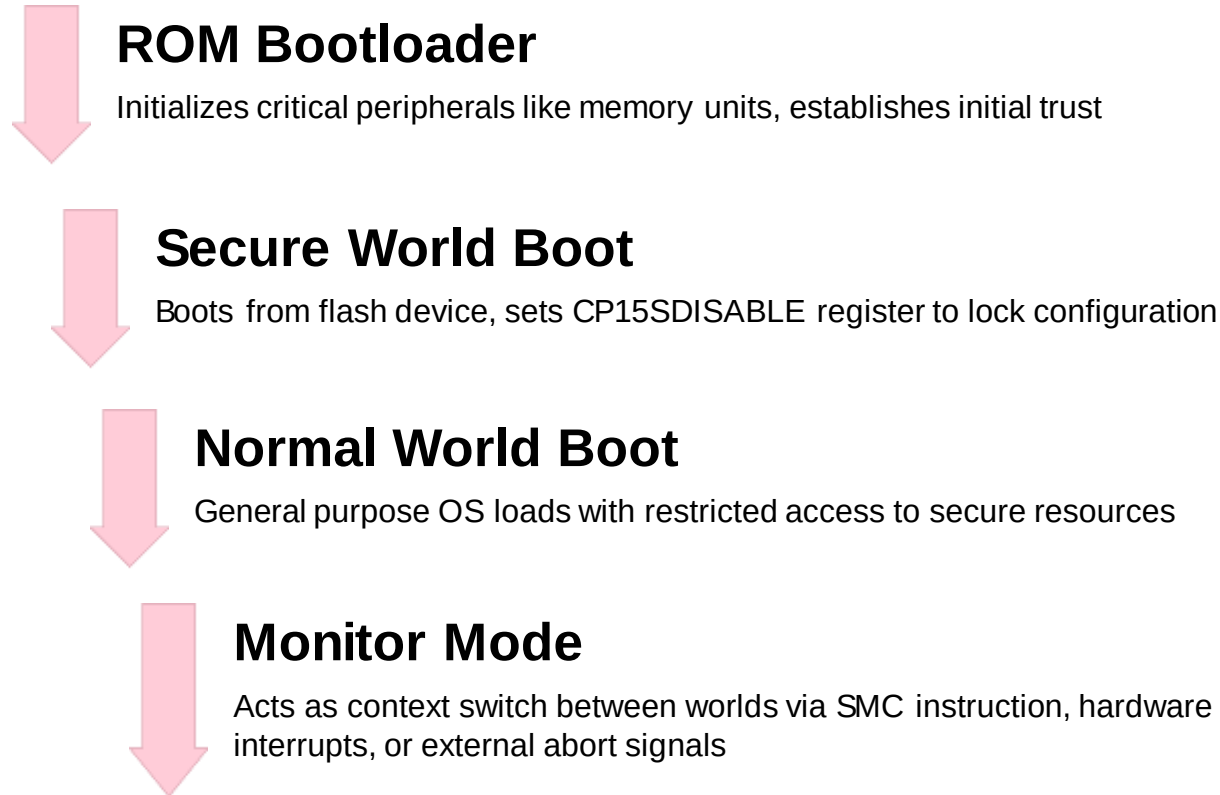
Context: Connection from CA to the Trusted OS.

Session: Connection from CA to TA.

Command: Unit of Communication from CA to TA.

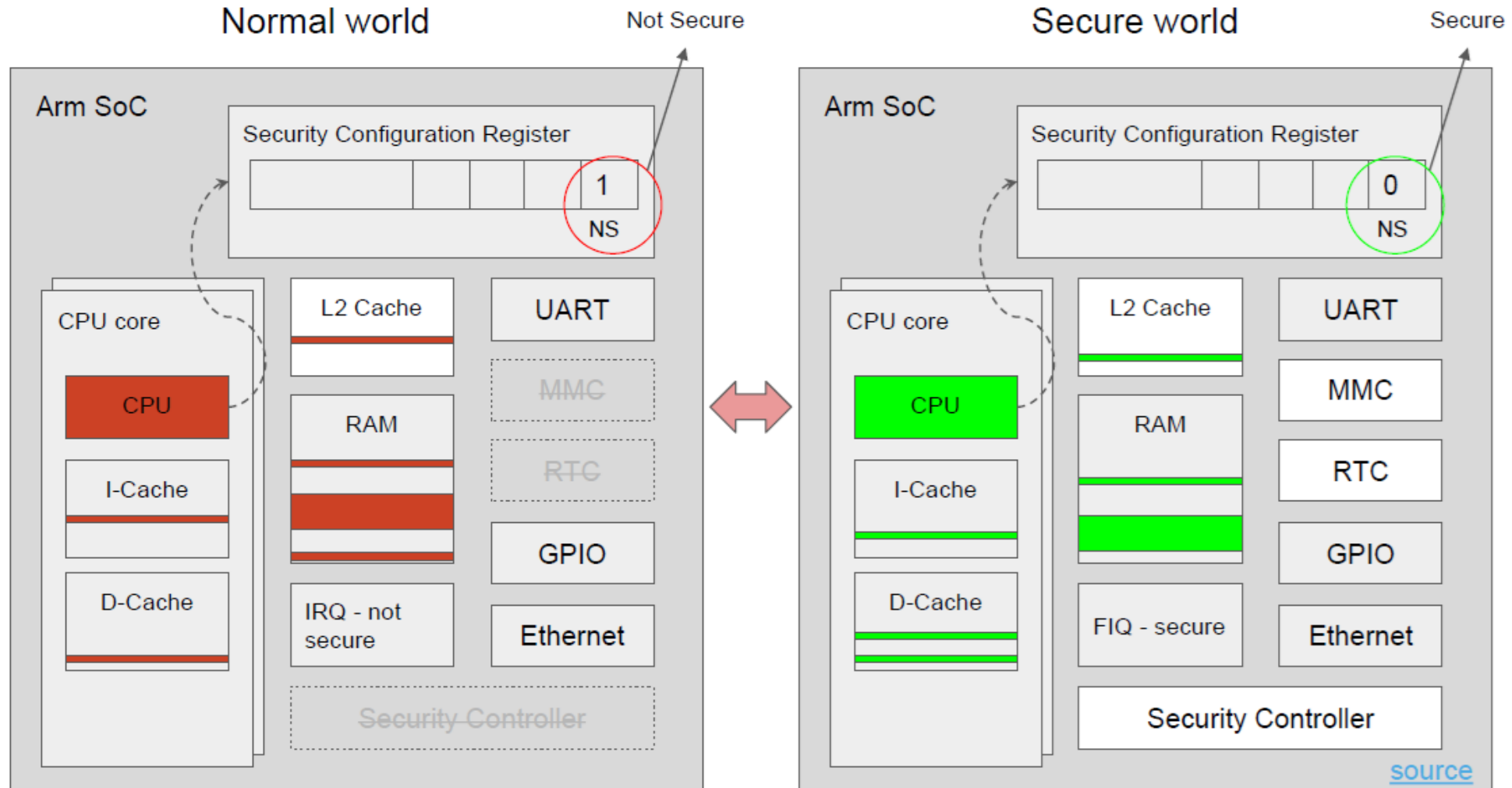
Shared Memory: Shared buffers allocated by Client API or by CA and registered.

Booting Sequence



- Each boot step is cryptographically verified using vendor-specific public keys, building a chain of trust.
- One-Time Programmable (OTP) techniques generate unique keys for each device component.

Hardware Support



Memory and Interrupt Management

■ Memory Controllers

- TZASC

- TrustZone Address Space Controller allows dynamic classification of memory-mapped devices as secure or non-secure, enabling arbitrary partitions

- TZMA

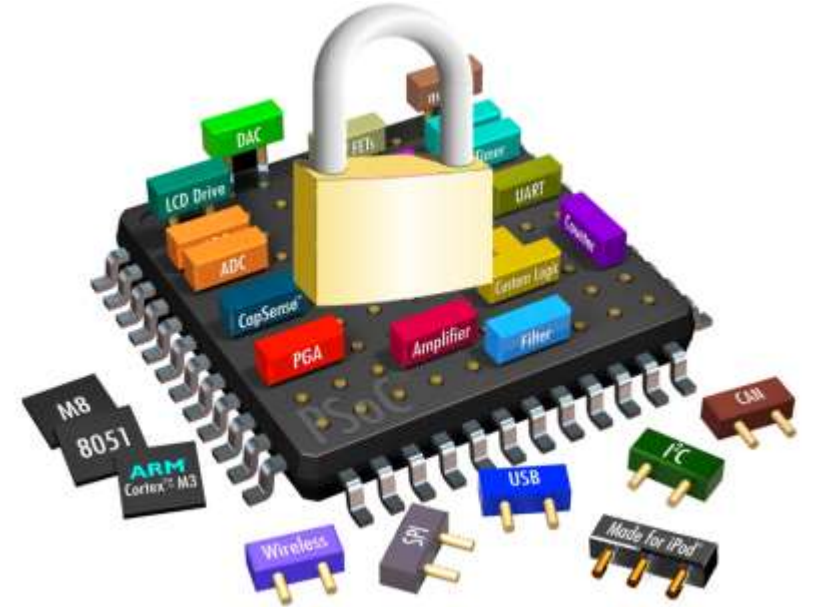
- TrustZone Memory Adapter divides on-chip static memory into secure and non-secure halves in 4KB multiples (max 2MB total)

■ Generic Interrupt Controller

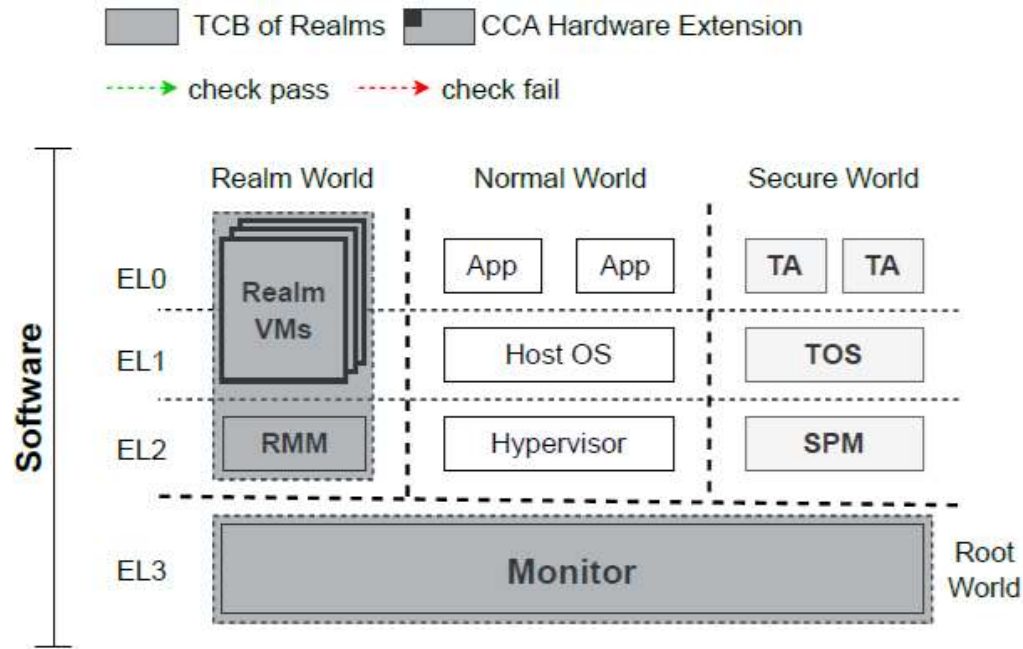
- The GIC handles secure and non-secure prioritized interrupts while preventing unauthorized access.
- To prevent denial-of-service attacks, it only handles interrupts in the lower half of the priority hierarchy.

Hardware Security

TEE Examples: ARM CCA
(Confidential Compute Architecture)

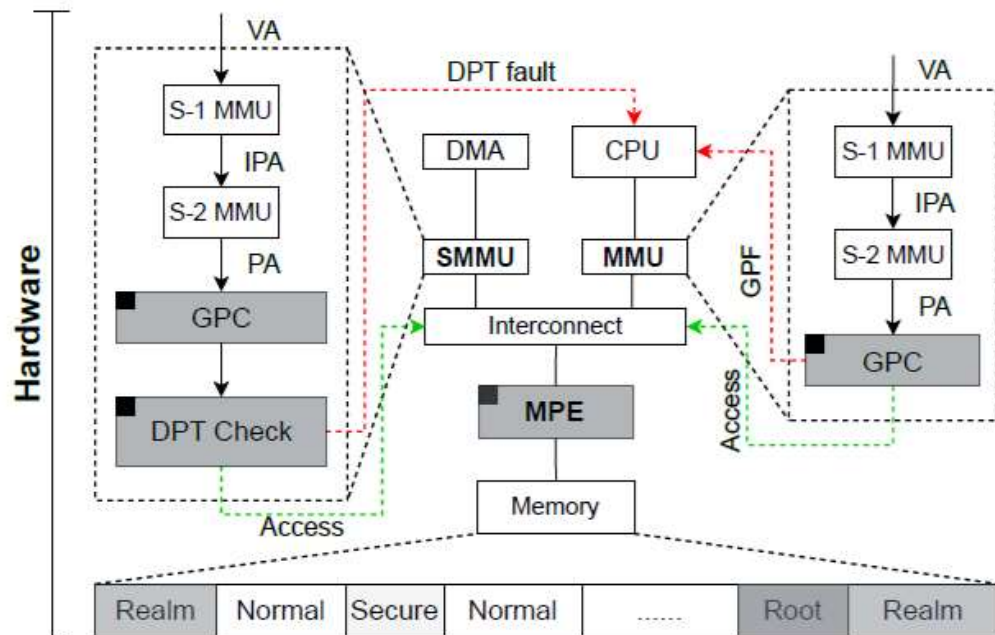


Four Worlds



- **Normal World**
 - Hosts standard operating systems and applications, similar to TrustZone's Normal world
- **Secure World**
 - Maintains compatibility with TrustZone's Secure world for existing trusted services
- **Root World**
 - New addition containing EL3 Monitor, responsible for trusted booting and world context switching
- **Realm World**
 - Breakthrough feature enabling third-party confidential VMs isolated from both Normal and Secure worlds

CCA Hardware



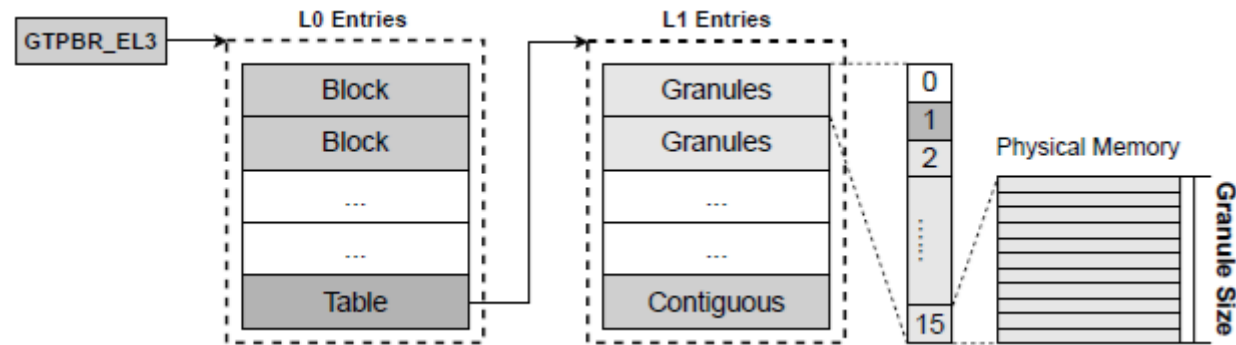
■ Granule Protection Check (GPC)

- CCA extends Memory Management Unit (MMU) with Granule Protection Check (GPC) to block invalid access.
- When translating Virtual Address (VA) to Physical Address (PA), GPC in MMU checks whether the current security state can access the target physical address.

■ Memory Protection Engine (MPE)

- MPE encrypts the entire memory space within the Realm world using a single key.
- To enhance security, CCA introduces a mechanism known as Memory Encryption Context (MEC), enabling a memory region in the Realm world to be assigned a unique encryption key.

Granule Protection Table



- MMU performs GPC based on the Granule Protection Table (GPT), an in-memory structure belonging to the Root world.
- In GPT, Granule Protection Information (GPI) indicates the associated world of granules.

CCA Advantages

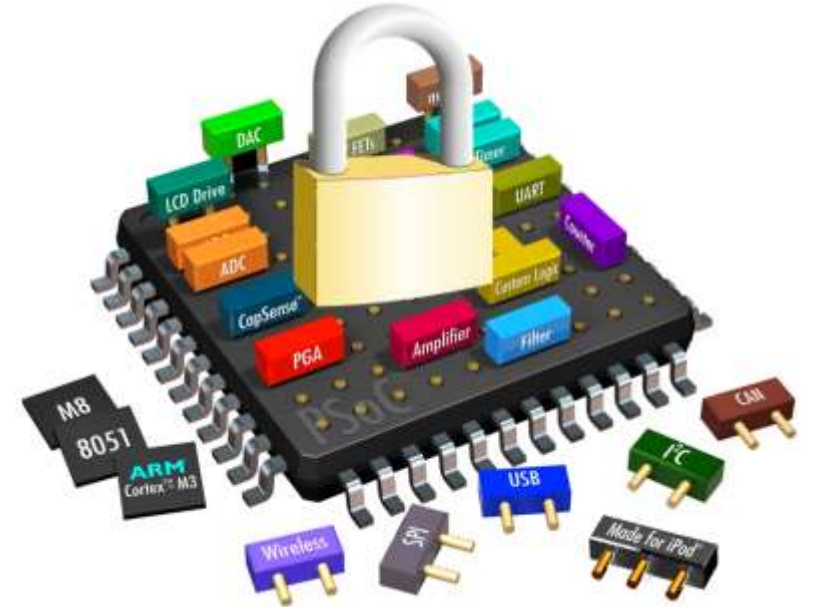
- Dynamic Allocation
 - CCA supports runtime memory transitions between worlds by modifying Granule Protection Information (GPI)
- Unlimited Regions
 - Unlike TZASC's limited region count, GPT manages unlimited memory regions
- Core-Specific Configuration
 - Each core can have independent memory partition settings
- Device Assignment
 - Fine-grained control binding DMA devices to specific Realm VMs
- Limitation
 - CCA's GPC doesn't support separate read/write permission configuration like TrustZone's TZASC

Security Enhancements in CCA

Criteria		TrustZone	CCA
Memory isolation	Access control for processors	√	√
	Access control for DMA	√	√
	Isolation between S-EL1/2 and EL3		√
	Level to configure	S-EL1/2	EL3
Memory encryption	Hardware-assisted encryption		√
Peripheral isolation	Access control for processors	√	√
	Access control for DMA	√	√
	Level to configure	S-EL1/2	EL3
Interrupt isolation	Individual interrupt for TEE	√	
Attestation	Hardware-assisted attestation		√

Hardware Security

TEE Examples: IBM PEF
(Protected Execution Facility)



Three Key Challenges in TEE Adoption

Balancing Isolation & Confidentiality

Server-class processors often link isolation and confidentiality, making TEEs expensive to operate.

Application Changes

Current implementations introduce many new security components requiring large application changes or greater trust.

Life Cycle Management

Managing secure entities in cloud deployments remains cumbersome due to attestation, secret provisioning, and integration complexity.

Protected Execution Facility (PEF)

- PEF provides a virtual machine-based TEE for OpenPOWER systems. It introduces the Protected Execution **Ultravisor**, new firmware managing hardware-enforced access control in a more privileged state than the hypervisor.
- PEF is supported in the latest POWER9 chip and demonstrates that combining access control for isolation with cryptography for confidentiality creates an effective confidential computing approach.

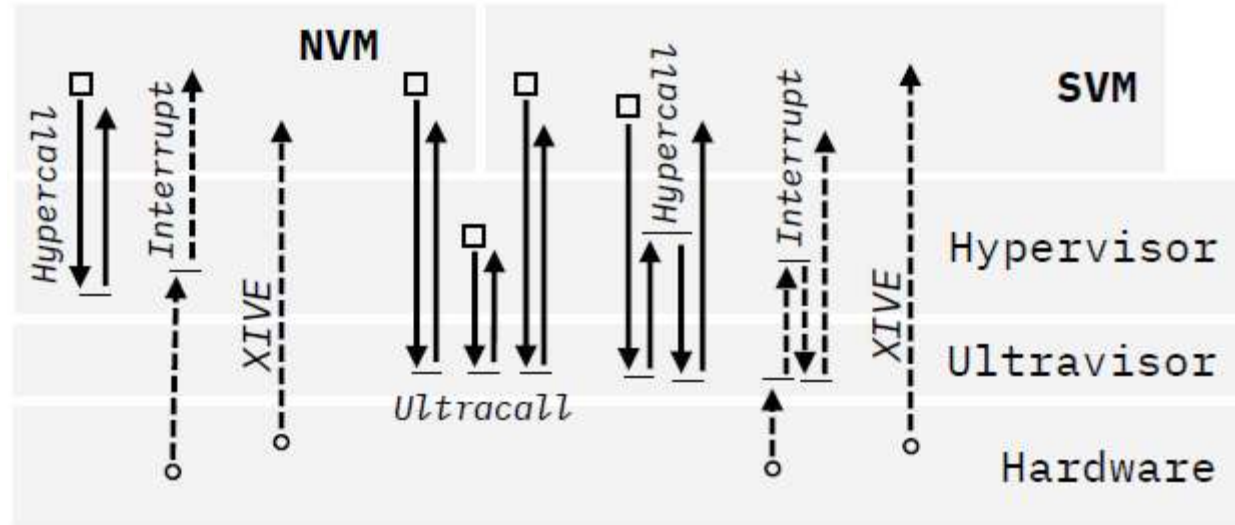
Secure State

Machine State Register (MSR)

Secure				Normal			
S	HV	PR		S	HV	PR	
1	0	0	privileged(OS)	0	0	0	privileged(OS)
1	0	1	problem	0	0	1	problem
1	1	0	ultravisor	0	1	0	hypervisor
1	1	1	(reserved)	0	1	1	problem (HV)

- PEF adds a new CPU state called secure state, more privileged than hypervisor state.
- The Ultravisor firmware manages all security-related hardware functions exclusively in this state.
- The Secure bit (S) in the Machine State Register determines whether system runs in normal or secure state.
- Secure state enables Protected Execution Ultravisor and Secure VMs.

Ultravisor



- Maintaining the **isolation** and **security** of the computation and associated data is the sole objective of the Ultravisor.
- System management continues to be the responsibility of the hypervisor.
- The hypervisor uses ultracalls to continue managing security sensitive facilities.
- In secure state, all hypervisor calls are delivered to the Ultravisor.
- Asynchronous interrupts are delivered to either the SVM or to the Ultravisor.

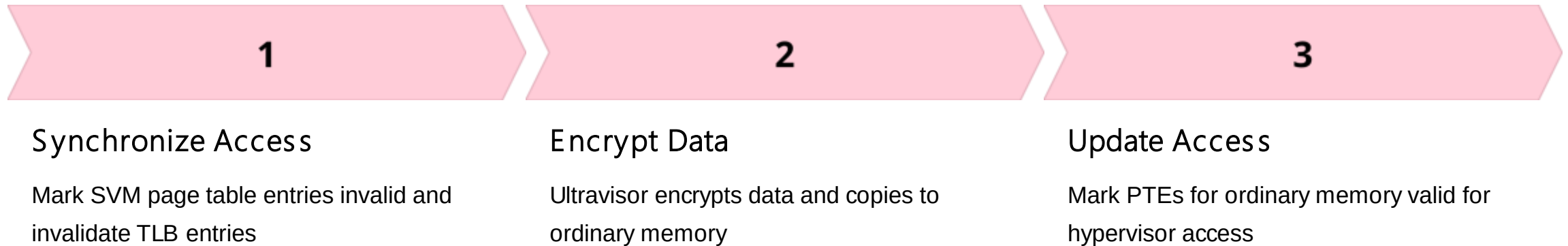
Access Control Architecture

- PEF uses security domains to enforce access control.
 - The hypervisor and normal VMs share one domain in normal memory.
 - Each Secure VM gets its own domain in secure memory.
- This provides bidirectional protection:
 - SVMs are protected from hypervisor and each other, while the hypervisor is protected from all SVMs.

Secure Memory Implementation

- Secure memory identified by MEMSM bit. In POWER9, implemented using high-order address bit RA(15).
- Hardware enforces that only processes in secure state can access secure memory.
- The Ultravisor controls the Partition Table Control Register, pointing to partition tables in secure memory.
- This isolates SVMs from each other, NVMs, and the hypervisor.

Maintaining Confidentiality



When hypervisor pages or dumps SVM, Ultravisor protects confidentiality through encryption with integrity using GCM before making data available.

Platform Verification Process

- For an SVM to start securely, three conditions must be met:
 - Platform must be verified
 - SVM authorized to run
 - SVM integrity verified
- All verification happens during the Enter Secure Mode (ESM) ultracall.
- Platform verification uses TPM and involves accessing the symmetric seed protecting the ESM operand and verifying firmware is in correct state.

Lockbox

What is a Lockbox?

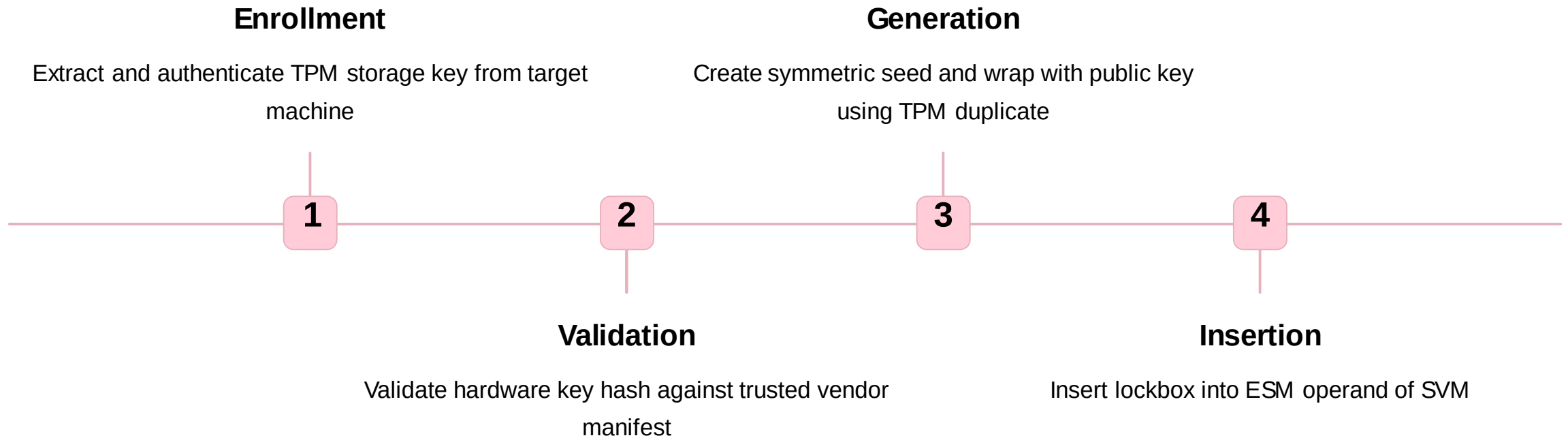
A symmetric seed wrapped in a public key associated with a private key in the target system's TPM. If no lockbox exists, SVM is not authorized.

Verification Steps

TPM extracts symmetric seed from lockbox. Firmware state verified through TPM platform configuration register (PCR) 6 value.



Generating Lockbox



Local Attestation

Local attestation verifies SVM integrity using information in the ESM operand. If Ultravisor has access to symmetric seed, it generates HMAC and symmetric keys.



Generate Keys

Create HMAC key and symmetric key from seed



Verify Integrity

Check ESM operand payload block integrity



Extract Information

Use symmetric key to get integrity data

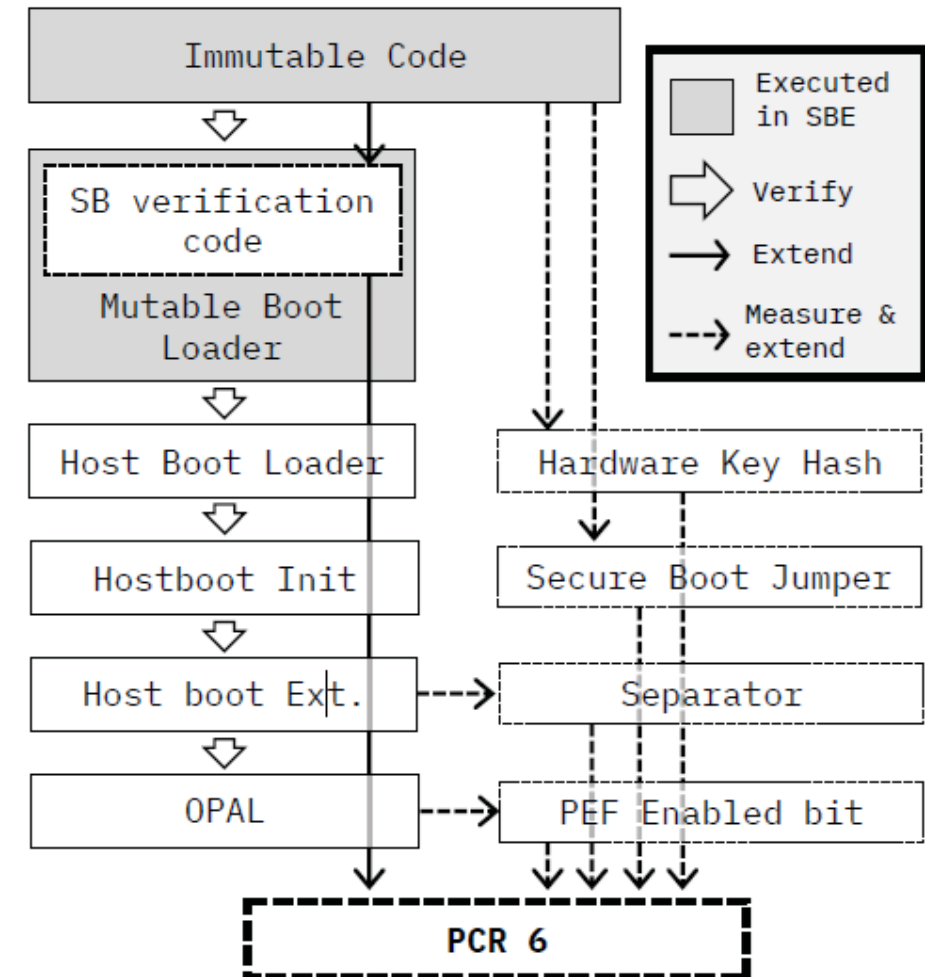


Validate Components

Verify kernel, command line, initramfs, RTAS

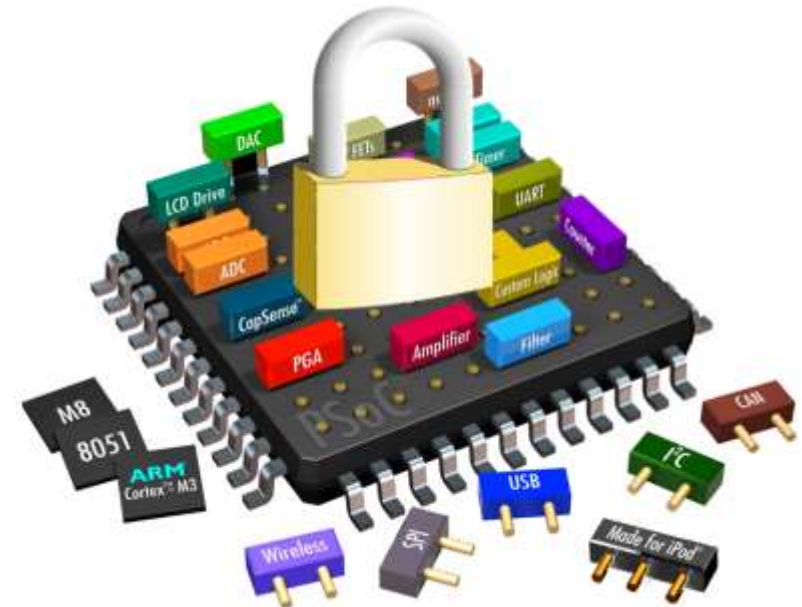
Hardware State Verification

- PEF relies on secure and trusted boot of the firmware stack.
- Only firmware signed by the owner (represented by hardware key hash) will be loaded.
- Secure boot verifies firmware validity, while trusted boot records information for state validation.



Hardware Security

TEE Examples: Keystone



Motivation

The Problem

Current trusted execution environments (TEEs) from major vendors—Intel SGX, ARM TrustZone, AMD SEV—represent **fixed** design points. Each makes inflexible choices in deployability, trusted computing base (TCB), and threat models.

Platform providers and developers are locked into limited possibilities, unable to customize for their specific needs. SGX's 128 MB memory limit causes drastic slowdowns in large-scale applications. Proprietary ISAs make modifications nearly impossible.

The Vision

We need **customizable TEEs**—a paradigm shift similar to Software Defined Networking. Hardware manufacturers provide baseline standards, platform providers customize implementations, and developers choose features that match their requirements.

This enables quick prototyping, faster fixes, adaptation to new threats, and use-case specific deployment without rebuilding from scratch.

Core Design Principles

01

Hardware-Enforced Memory Isolation

Flexible isolation as a native primitive, customizable at runtime by privileged mode

03

Modularity Throughout

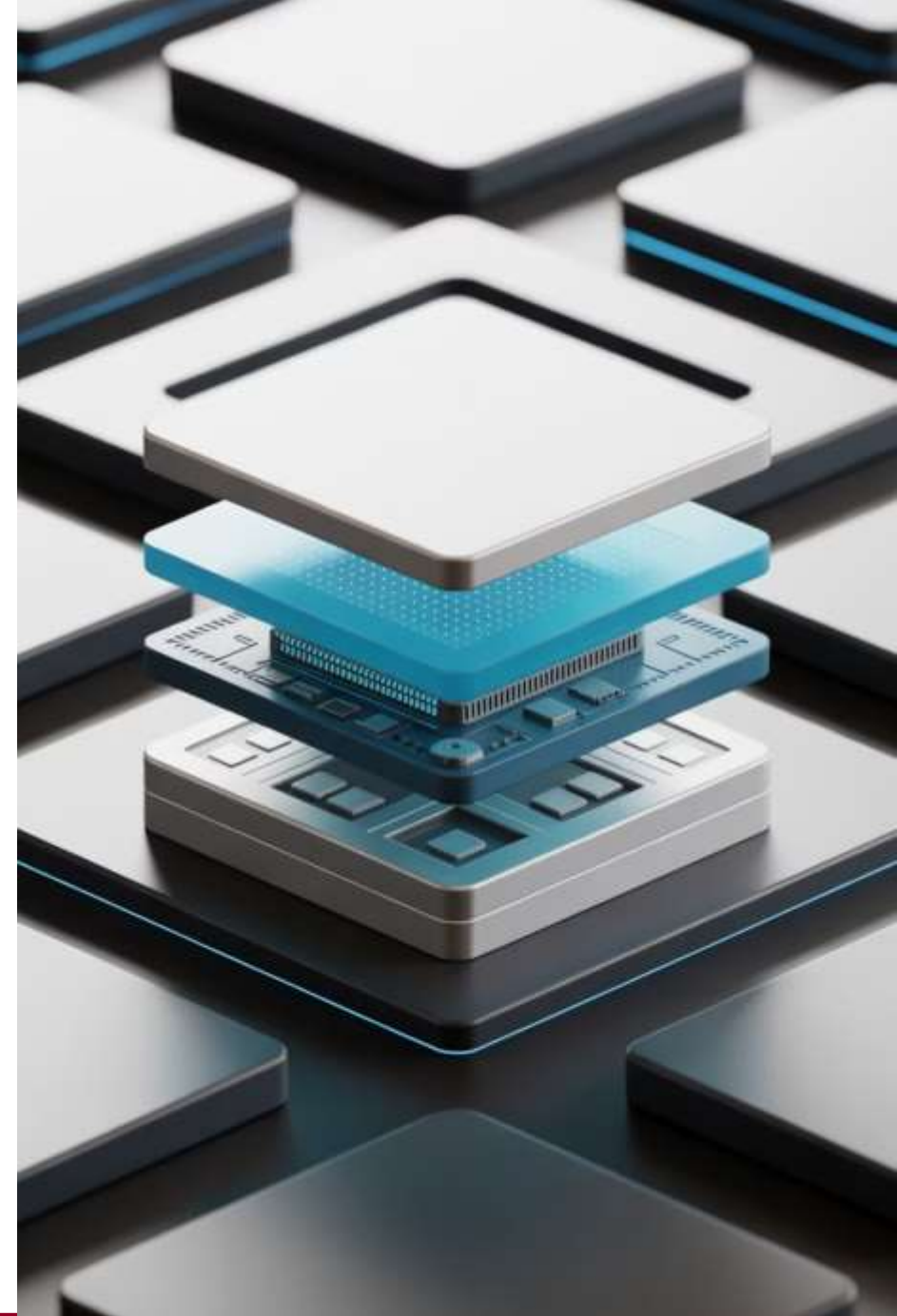
Independent layers with clean abstractions, maintaining compatibility with existing privilege levels

RISC-V provides these abstractions through M-mode (programmable highest privilege), hardware interrupt delegation, and PMP (runtime memory isolation). This makes RISC-V a natural fit for customizable TEEs.

02

Programmable Trusted Layer

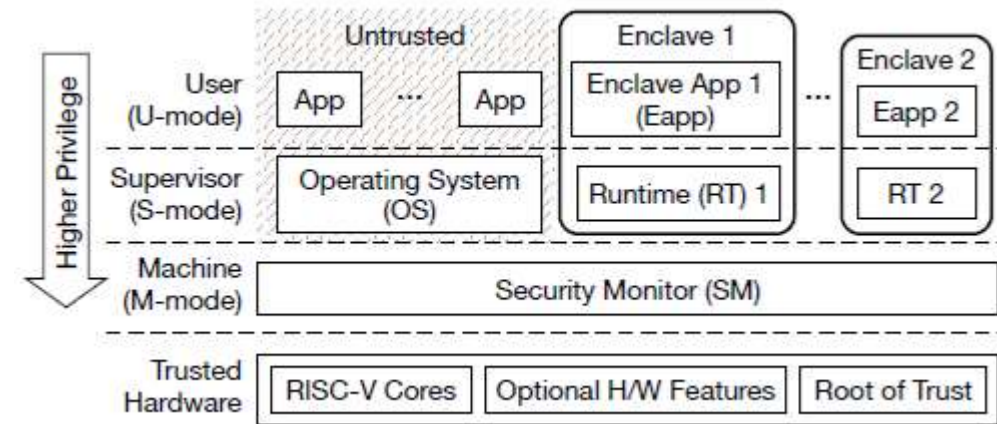
Software layer with higher privileges than OS, managing isolated memory regions with minimal TCB



Customization in Action

- Platform Support
 - Unlike vendor-specific TEEs, Keystone supports various workloads across embedded devices, mobile platforms, and cloud servers without platform lock-in
- TCB Size
 - Modular design allows minimal TCB for simple applications or rich functionality for complex ones—same database can run with thousands or millions of lines of code
- Threat Models
 - Flexible defense mechanisms adapt to physical attackers, software attackers, side-channel attacks, or denial-of-service—enable only what you need

System Architecture

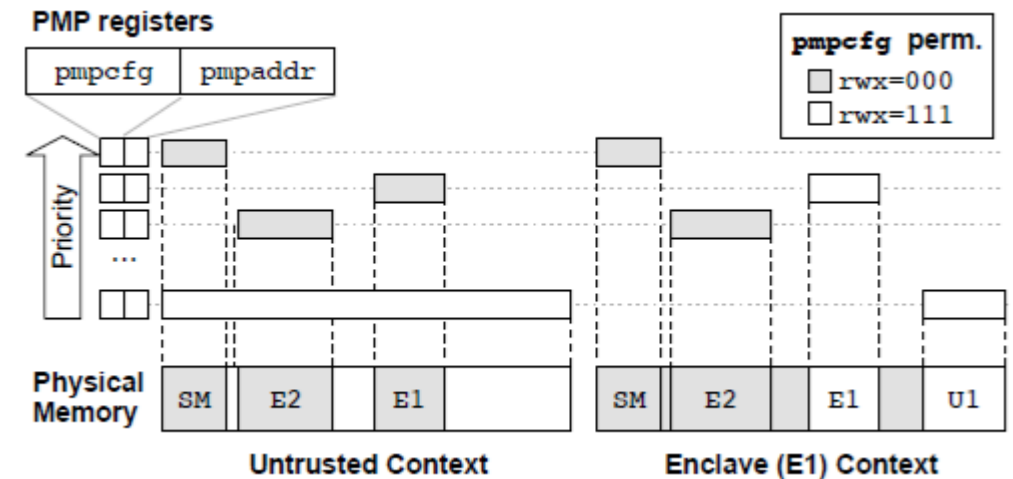


- Security Monitor (SM)
 - Executes in M-mode, enforces TEE guarantees across the platform.
 - Configurable by the platform generator.
- Runtime (RT)
 - Executes in S-mode within enclave, manages lifecycle and memory.
 - User-specified and configurable per application needs.
- Enclave Application
 - Executes in U-mode, isolated from untrusted OS.
 - Can be unmodified existing applications with appropriate RT.

Memory Isolation with RISC-V PMP

Keystone leverages RISC-V Physical Memory Protection (PMP) for flexible, dynamic memory isolation. PMP allows M-mode to restrict S-mode and U-mode access to customizable physical memory regions.

- 1 Flexible Regions**
Multiple contiguous enclave regions proportional to PMP registers—no single large shared region
- 2 Arbitrary Sizes**
Regions from 4 bytes to entire DRAM with page-aligned memory of any size
- 3 Dynamic Reconfiguration**
Create or release regions during execution instead of permanent occupation



The SM configures PMP entries to protect its own memory and enclave regions. During context switches, permission bits flip to allow enclave access while protecting OS memory, and vice versa.

TEE Primitives

- Secure Boot
 - Root-of-trust measures SM image, generates attestation key, signs measurement with hardware secret
- Secure Randomness
 - Hardware source or fallback methods (CPU jitter, DRAM decay) via SM SBI interface
- Remote Attestation
 - SM signs enclave measurement with attestation key, binds to secure channel construction
- Sealed Storage
 - Monotonic counters in SM memory, future support for encrypted content tied to enclave identity
- Trusted Timer
 - Non-writable hardware timer registers accessible via rdcycle instruction
- Secure Delegation
 - Safe trap handling without leaking enclave state or allowing arbitrary entry points